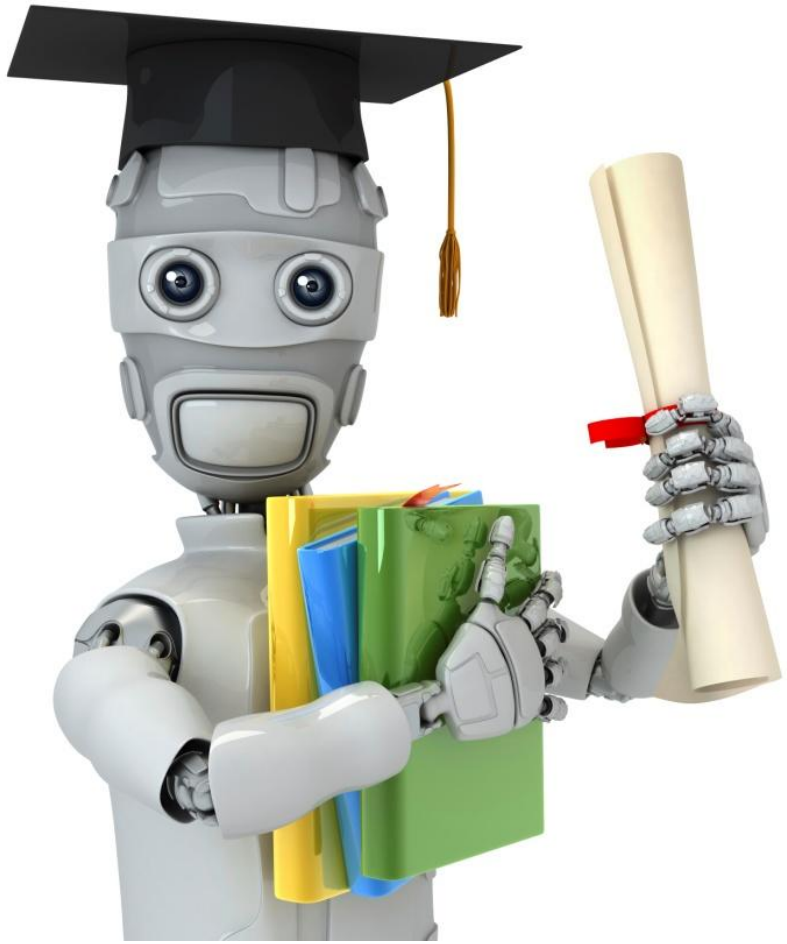
Lecture 10 – Linear Regression (Theoretical Foundation)

Dr. Sifat Momen

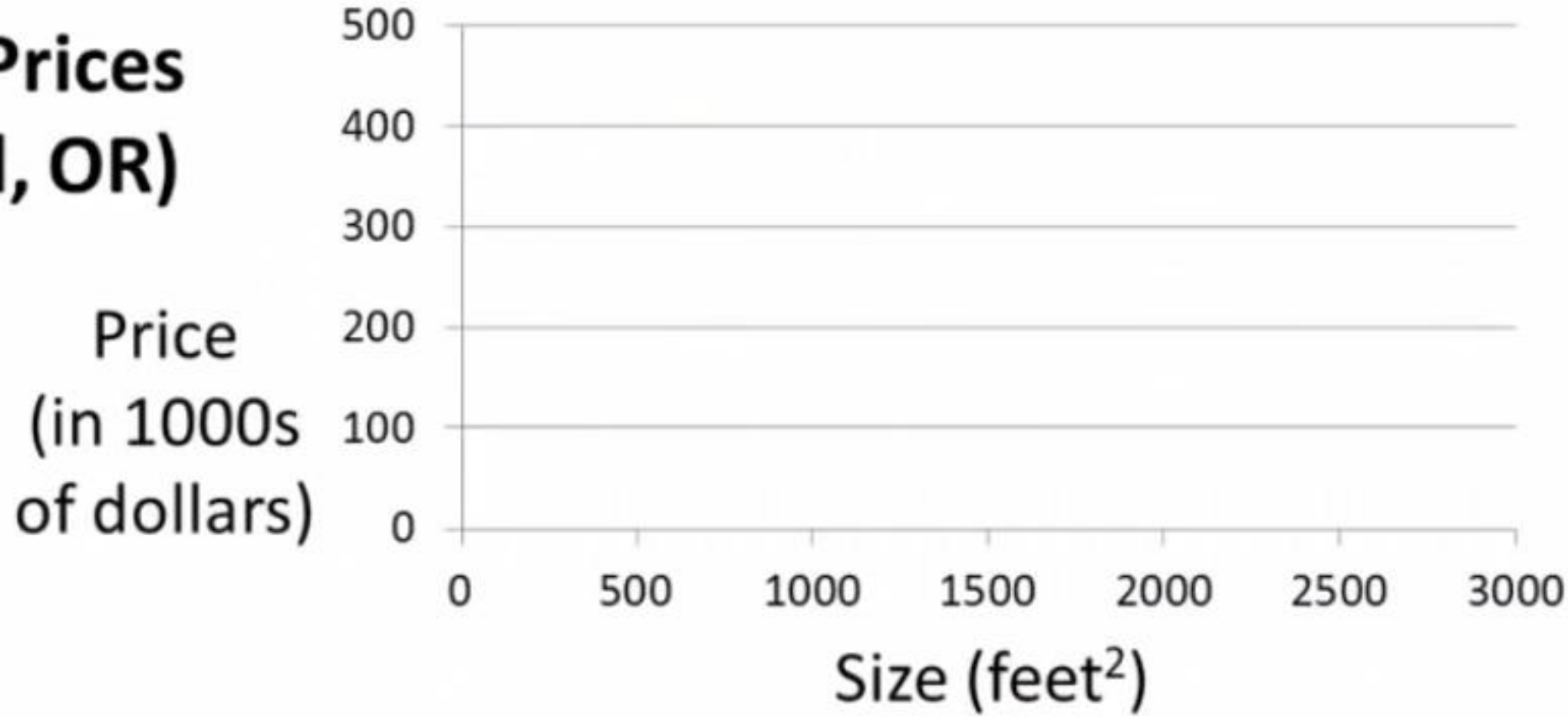


Machine Learning

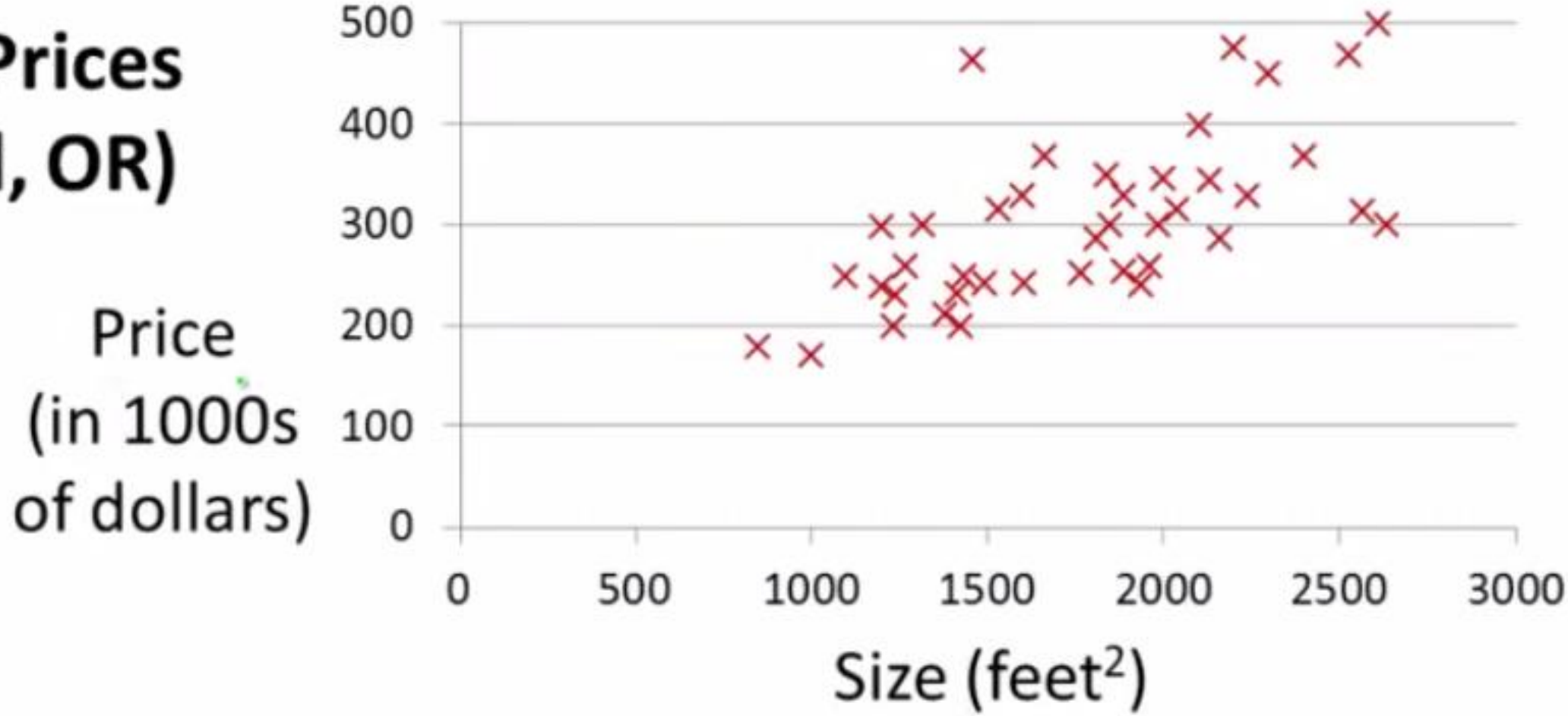
Linear regression with one variable

Model representation

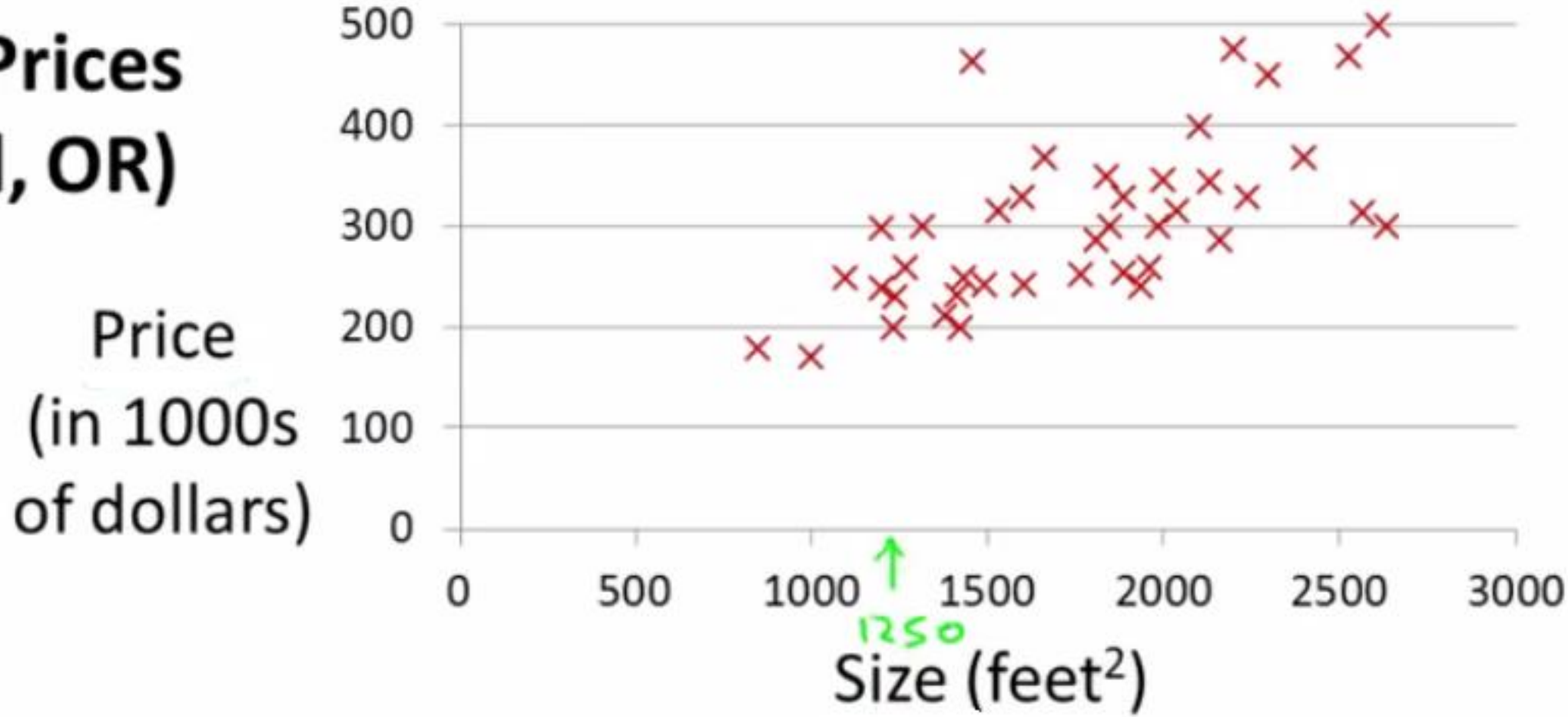
Housing Prices (Portland, OR)



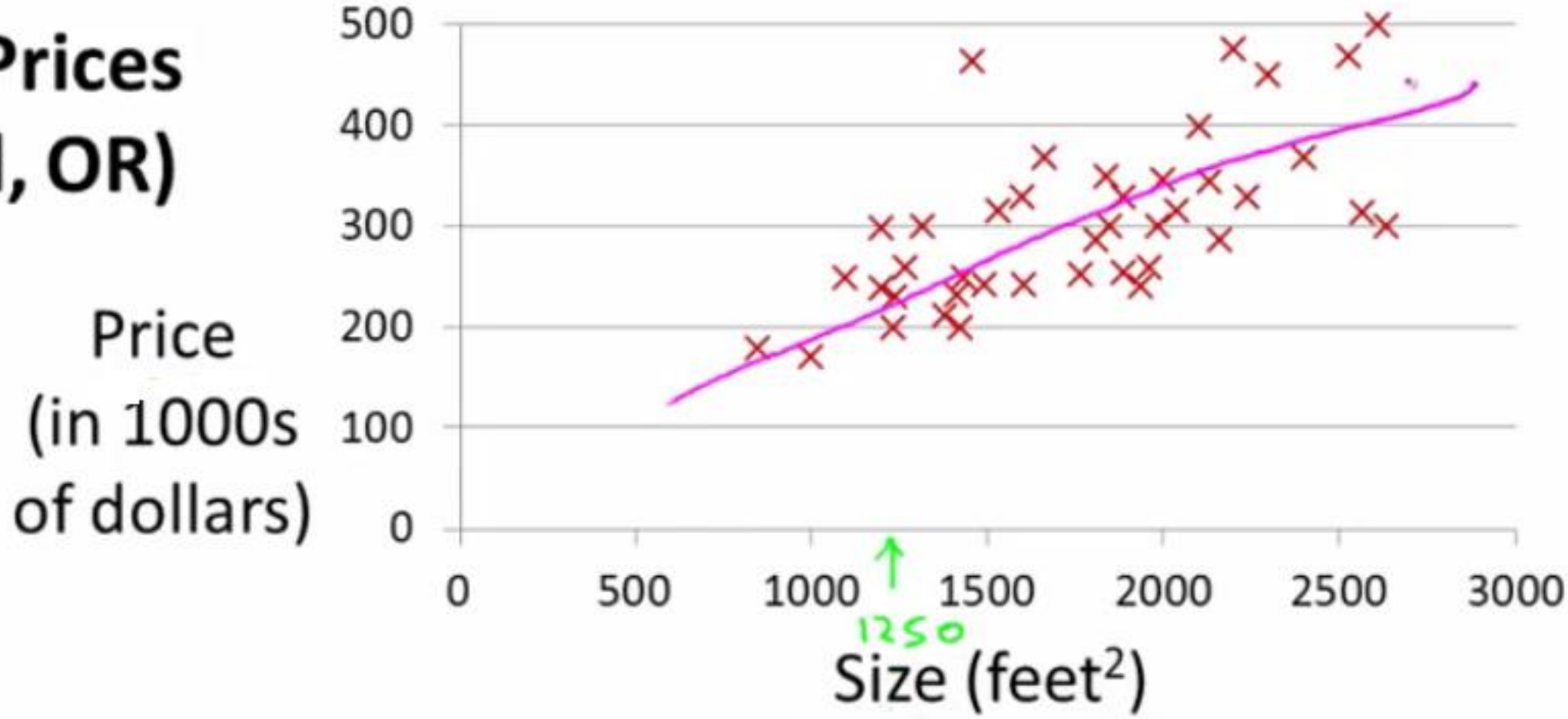
Housing Prices (Portland, OR)



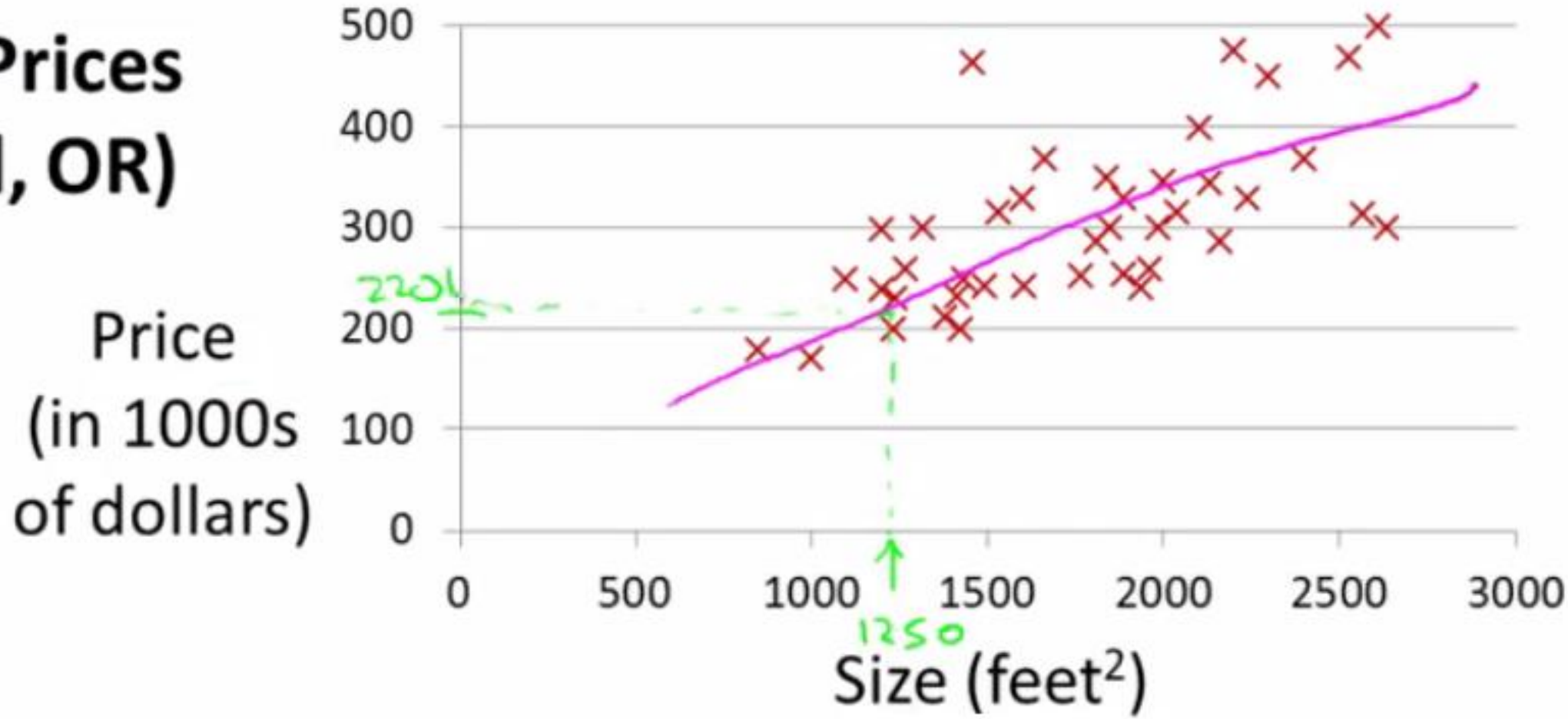
Housing Prices (Portland, OR)



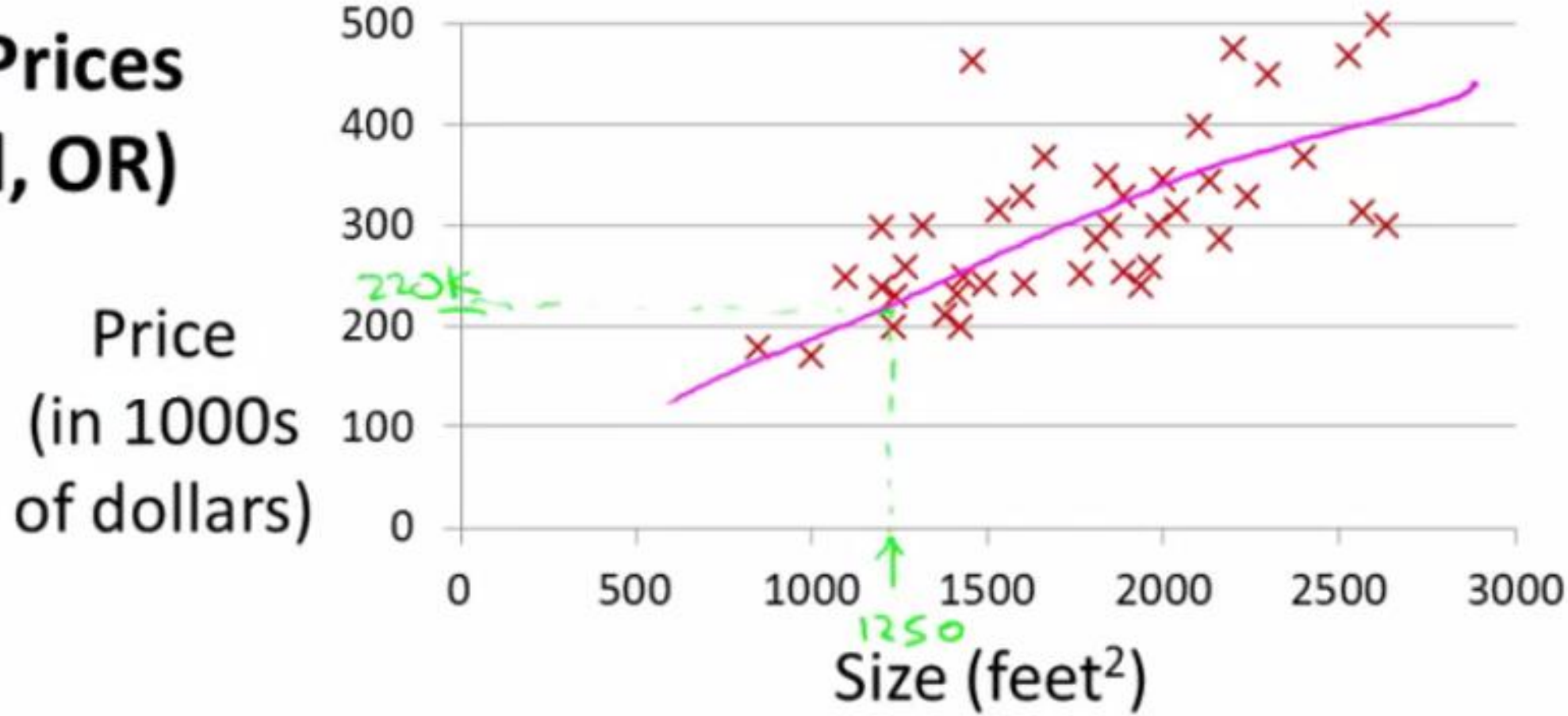
Housing Prices (Portland, OR)



Housing Prices (Portland, OR)



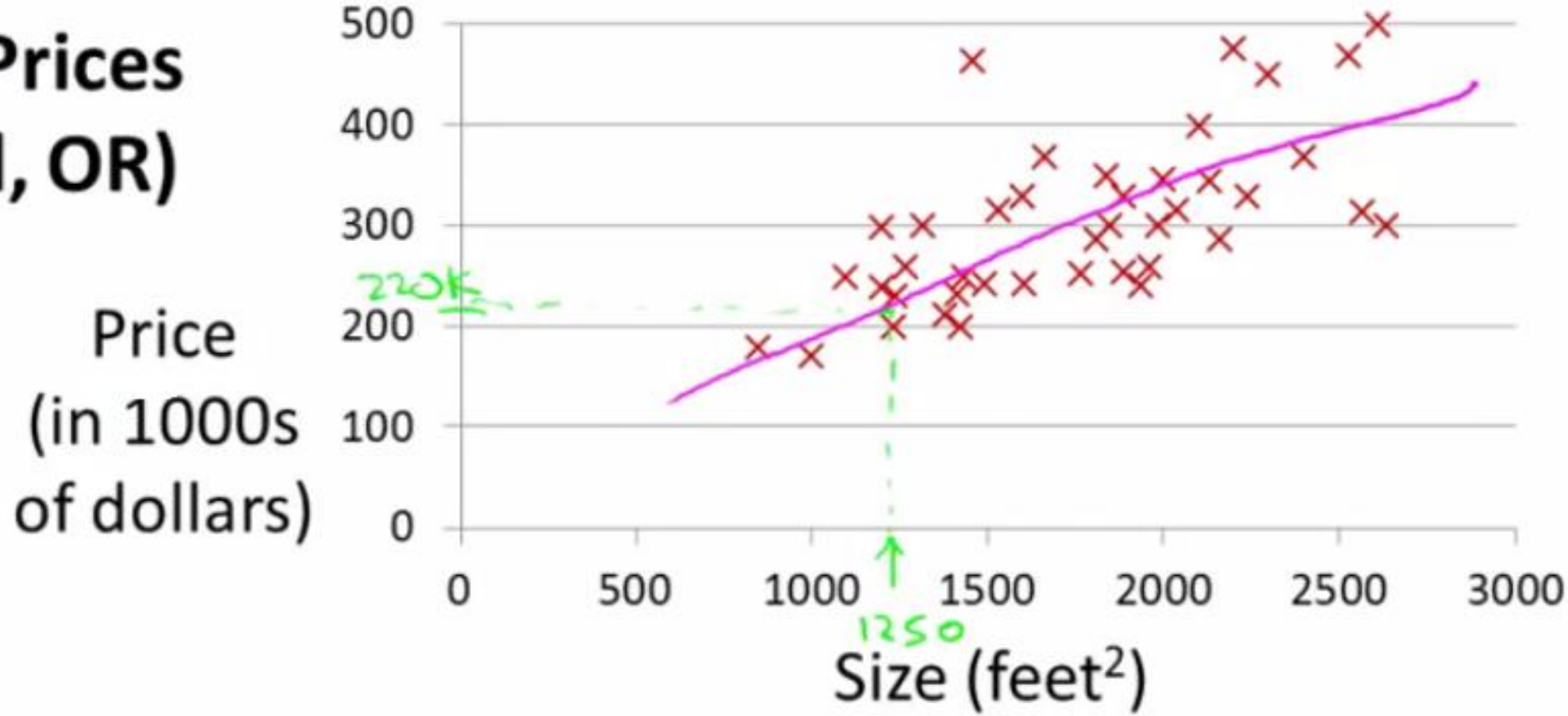
Housing Prices (Portland, OR)



Supervised Learning

Given the "right answer" for each example in the data.

Housing Prices (Portland, OR)



Supervised Learning

Given the "right answer" for each example in the data.

Regression Problem

Predict real-valued output

Training set of housing prices (Portland, OR)

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable

Training set of housing prices (Portland, OR)

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

(x, y)= One Training Example

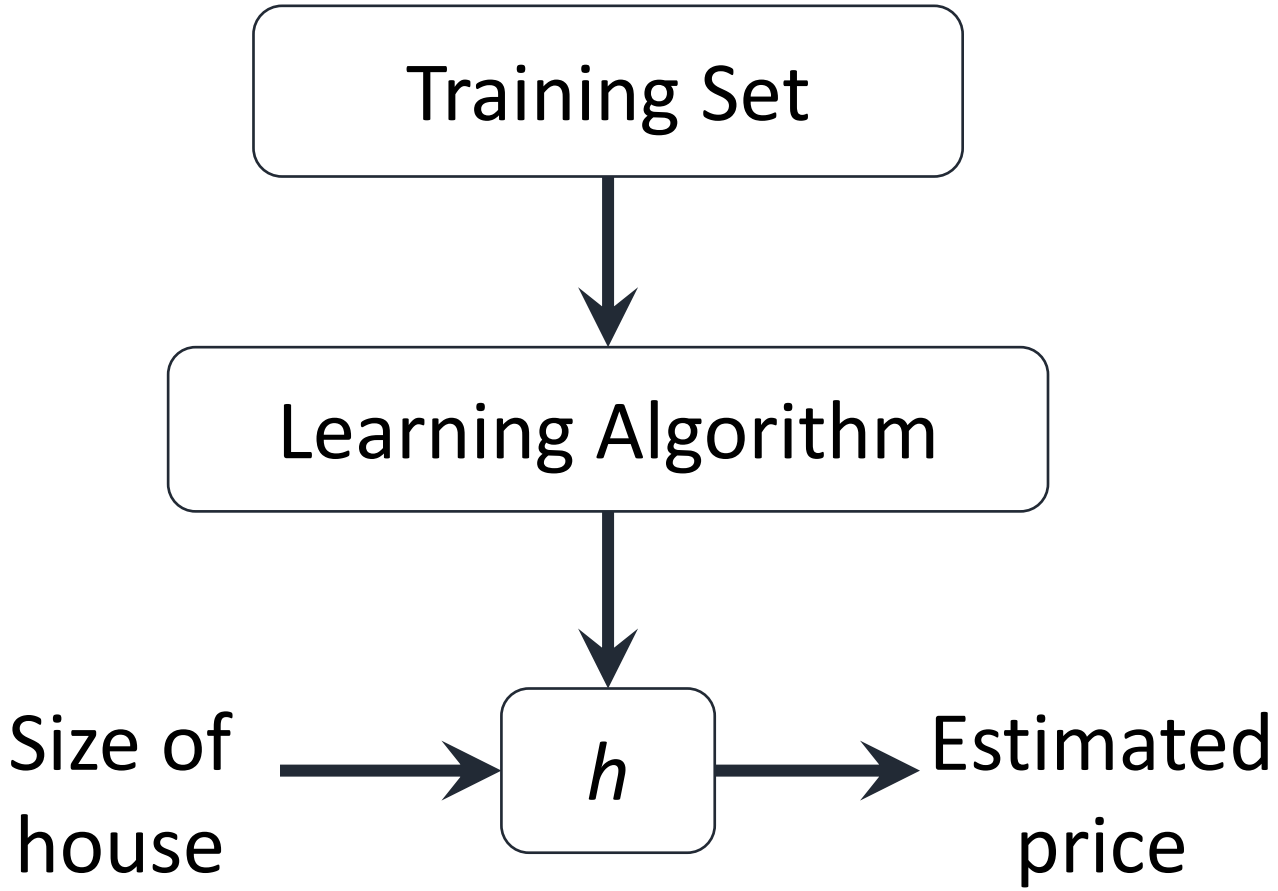
x (i), y (i)= ith Training example

x (1) = 2104

x (2) = 1416

y (1) = 460

Y (2) = 232



How do we represent *h* ?

$$h_\theta(x) = \theta_0 + \theta_1 x$$

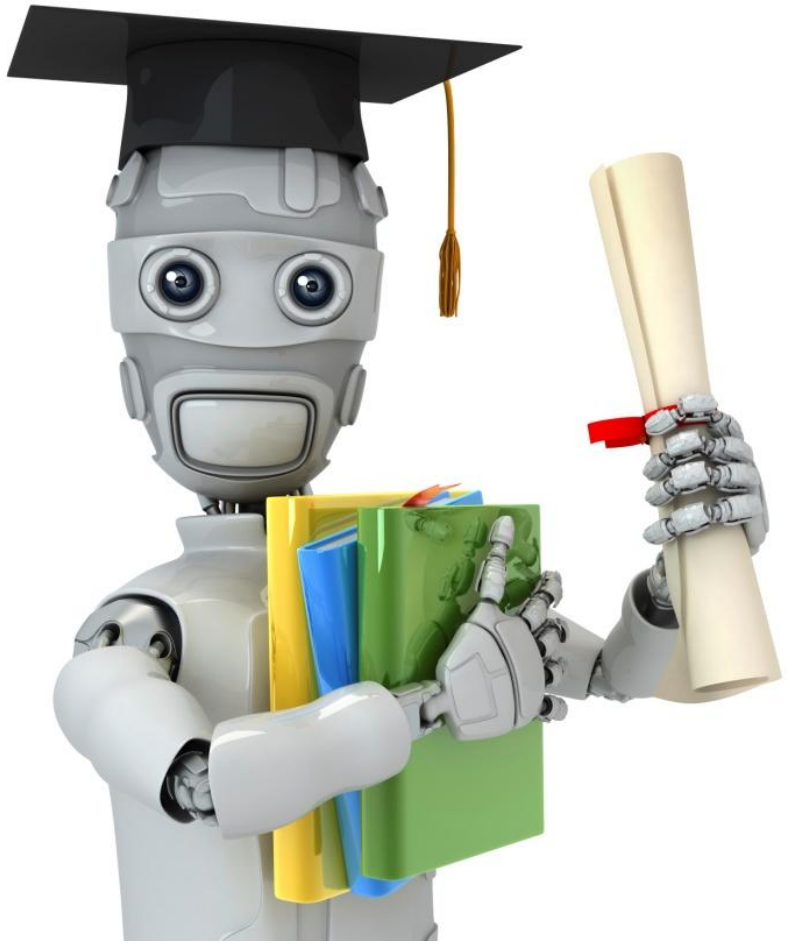
Linear regression with one variable.
Univariate linear regression.

$$h_\theta(x) = \theta_0 + \theta_1 x$$

That formula might look scary, but if you think about it it's nothing fancier than the traditional equation of a line, except that we use θ0 and θ1 instead of m and q. Do you remember?

$$f(x)=q+mx$$

In our humble hypothesis function there is only **one variable**, that is x. For this reason our task is often called **linear regression with one variable**. Experts call it also **univariate linear regression**, where univariate means "**one variable**".



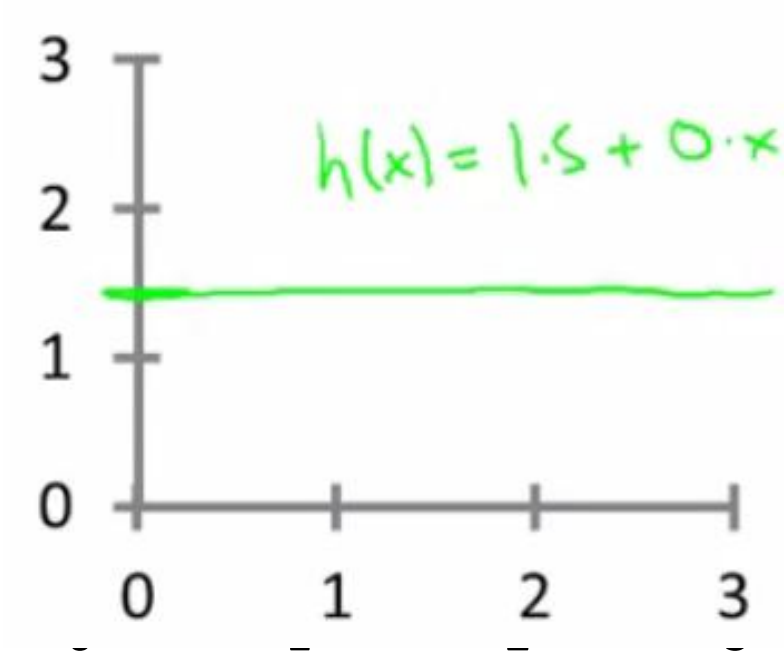
Machine Learning

Linear regression with one variable

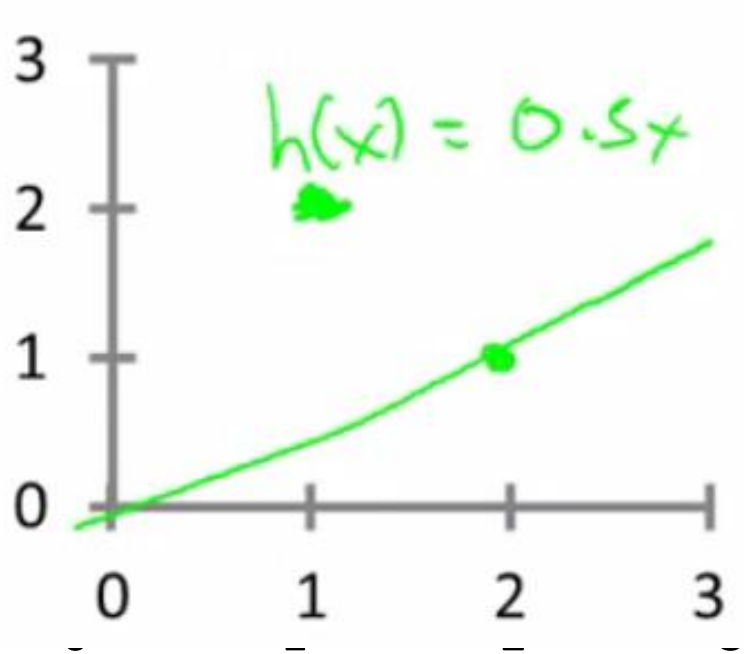
Cost function

A cost function lets us figure out how to fit the best straight line to our data.

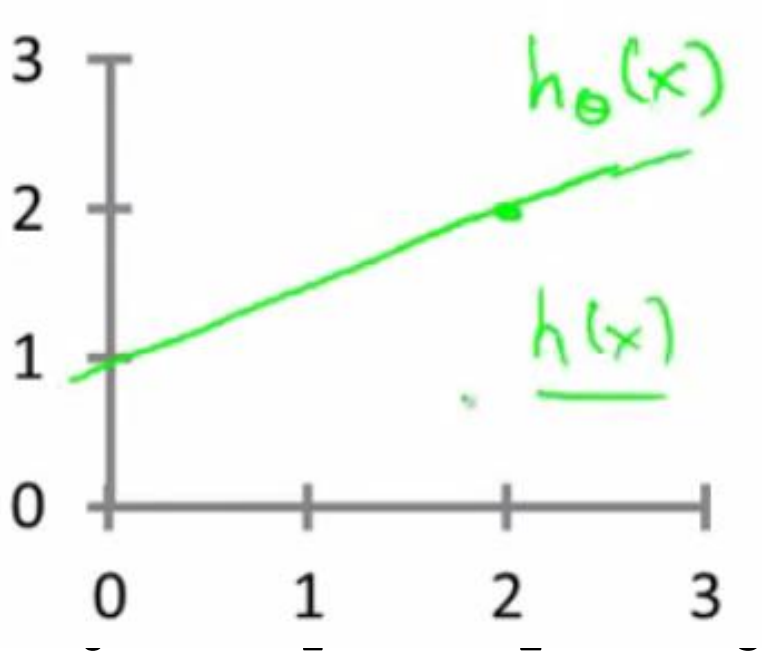
$$h_\theta(x) = \theta_0 + \theta_1 x$$



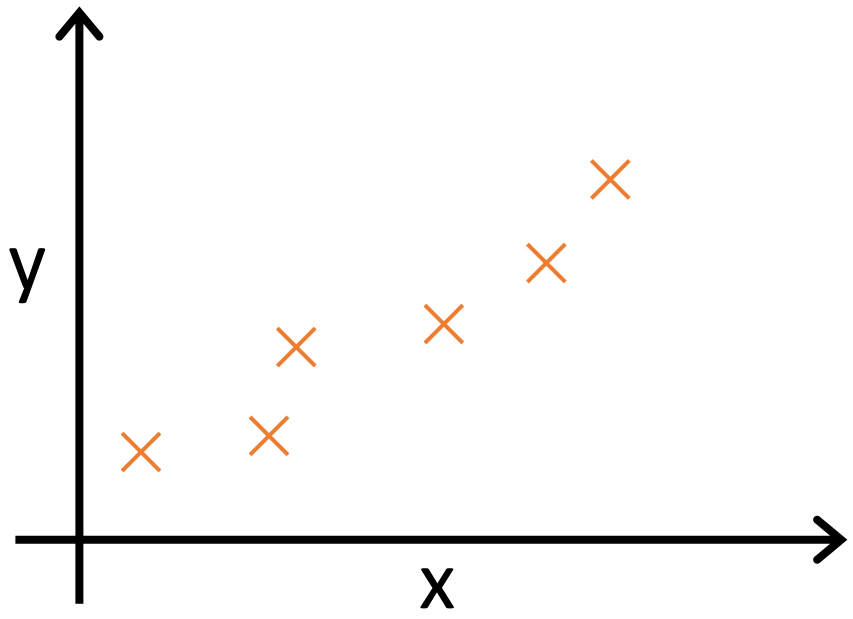
$\theta_0 = 1.5$
$\theta_1 = 0$



$\theta_0 = 0$
$\theta_1 = 0.5$

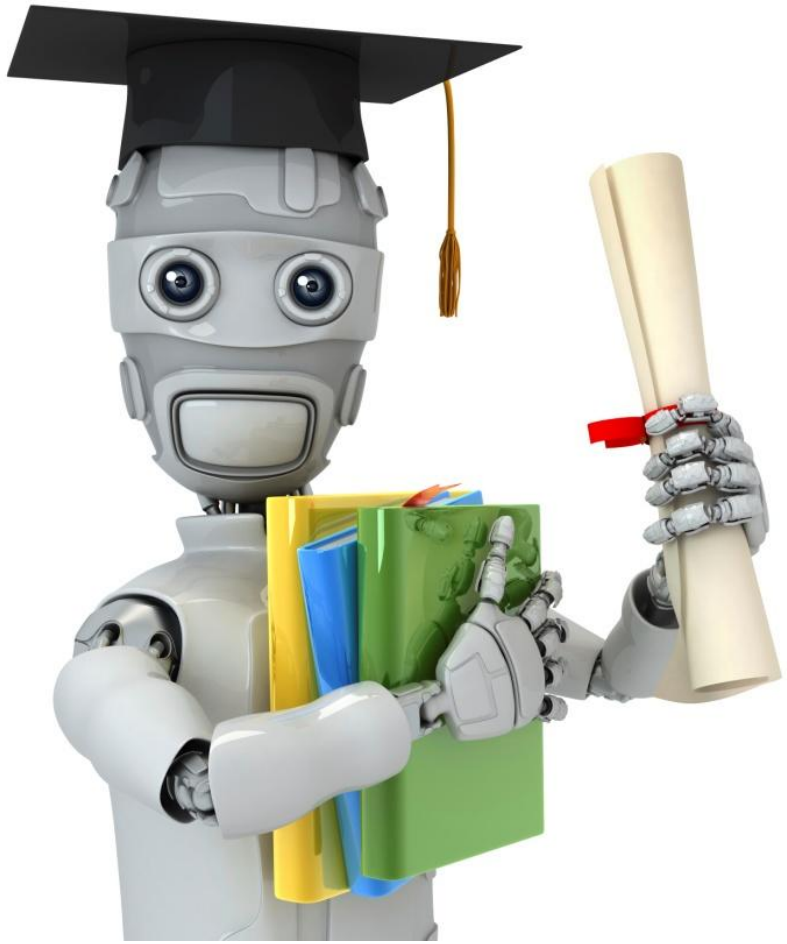


$\theta_0 = 1$
$\theta_1 = 0.5$



Idea: Choose θ_0, θ_1 so that
$h_\theta(x)$ is close to y for
our training examples (x, y)

$$J(\theta_0, \theta_1) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$



Machine Learning

Linear regression with one variable

Cost function intuition I

Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$

Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Goal: $\underset{\theta_0,\theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Simplified

$$h_\theta(x) = \theta_1 x$$

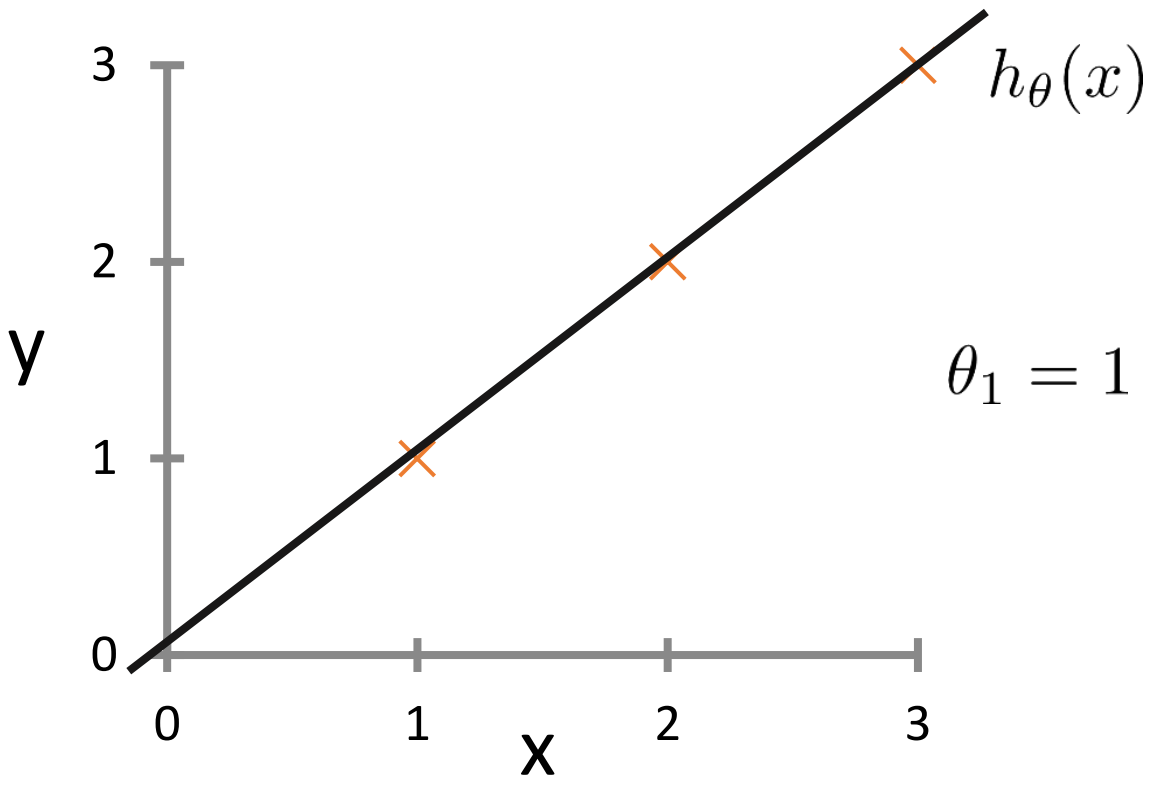
$$\theta_1$$

$$J(\theta_1) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

$\underset{\theta_1}{\text{minimize}}\ J(\theta_1)$

$h_\theta(x)$

(for fixed θ_1, this is a function of x)



So for example

$\theta_1 = 1$

$J(\theta_1) =$

$\theta_1 = 0.5$

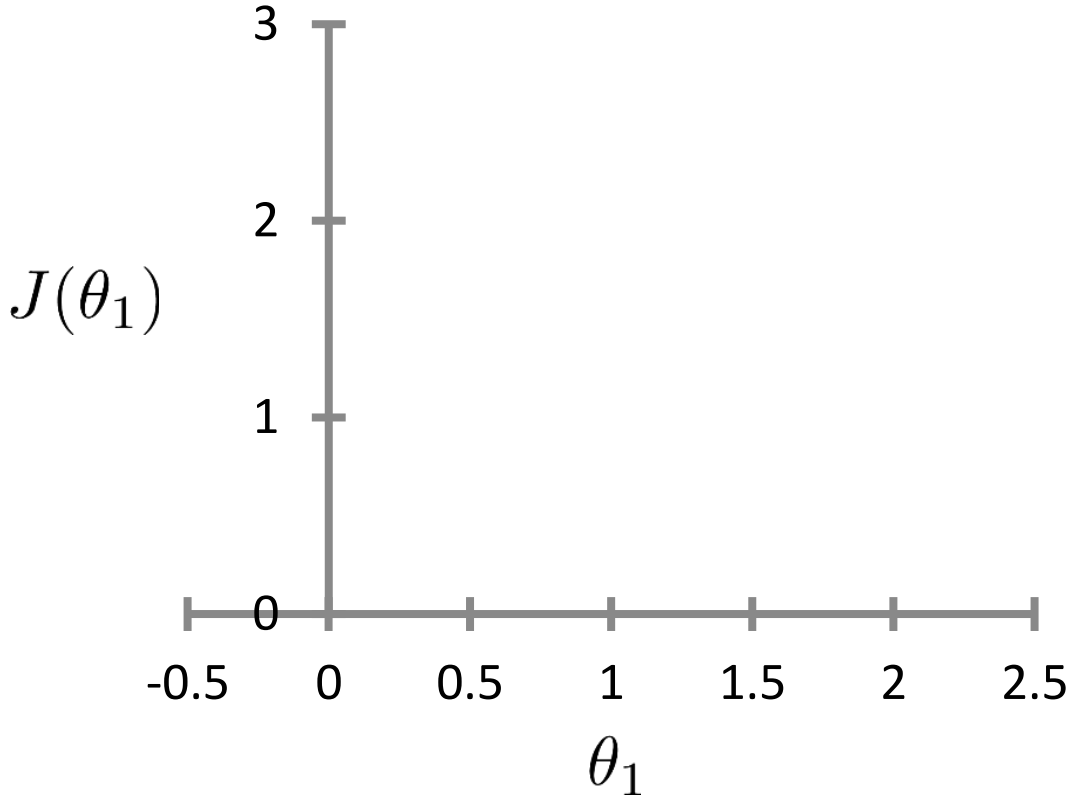
$J(\theta_1) =$

$\theta_1 = 0$

$J(\theta_1) =$

$J(\theta_1)$

(function of the parameter θ_1)



Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

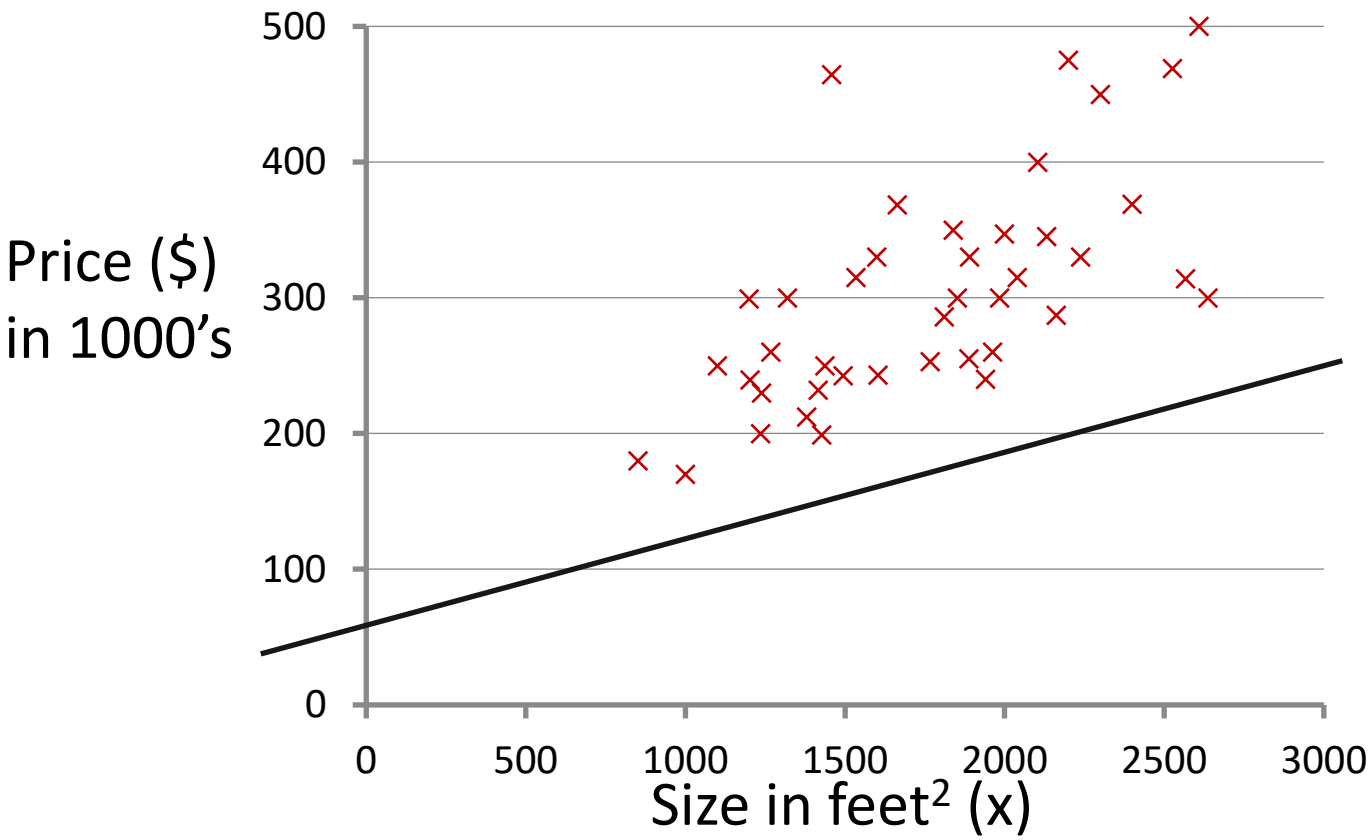
Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$

Goal: $\underset{\theta_0,\theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

$h_\theta(x)$

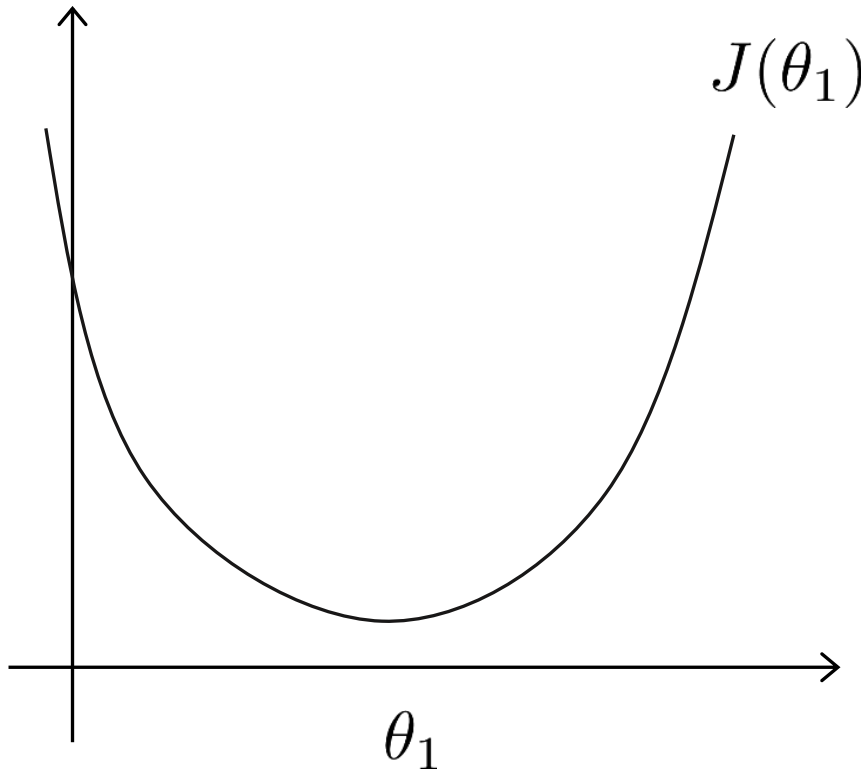
(for fixed θ_0, θ_1, this is a function of x)



$$h_\theta(x) = 50 + 0.06x$$

$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)

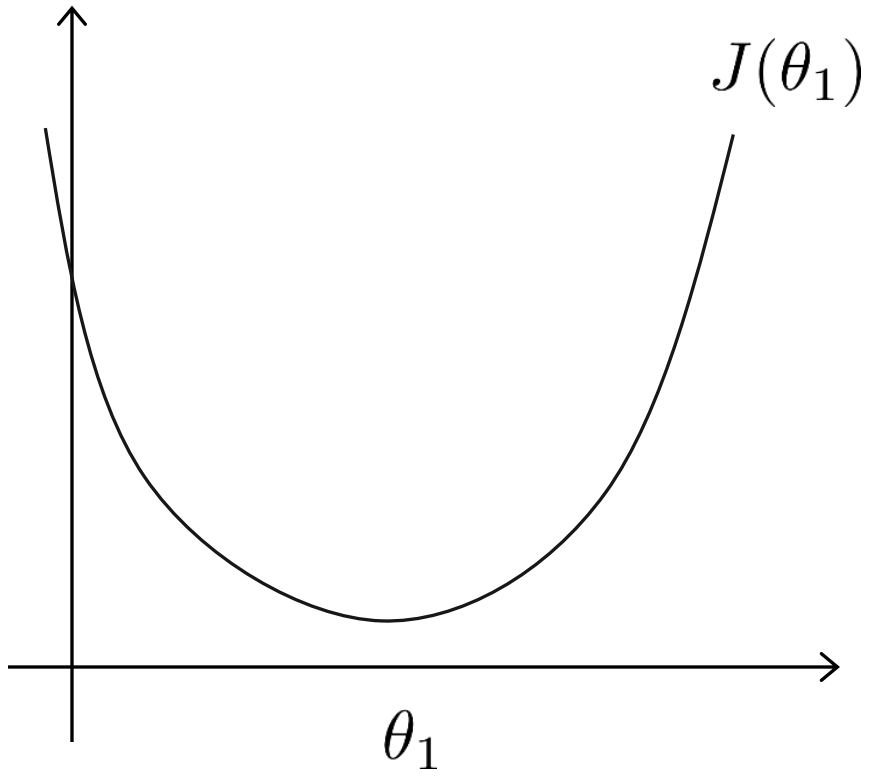


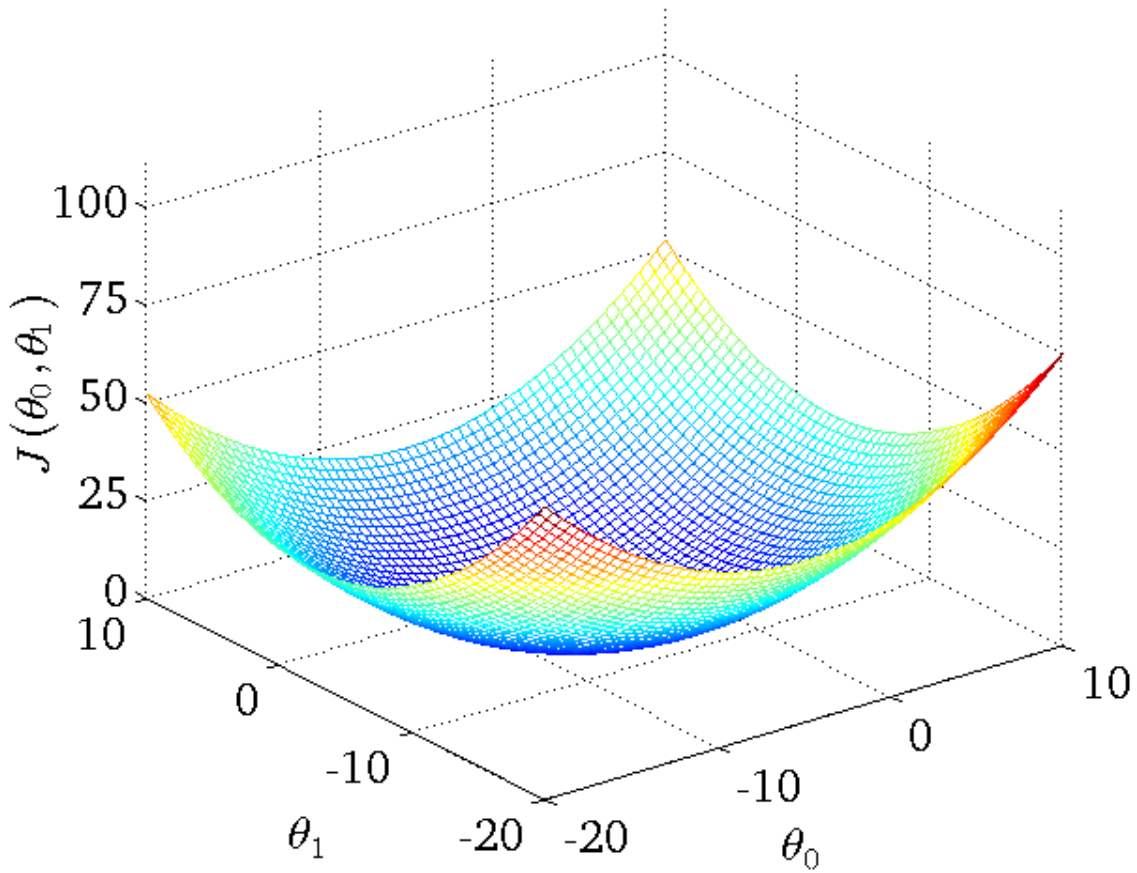
Previously we plotted our cost function by plotting

- θ_1 vs $J(\theta_1)$

Now we have two parameters

- Plot becomes a bit more complicated
- Generates a 3D surface plot where axis are
  - $X = \theta_1$
  - $Z = \theta_0$
  - $Y = J(\theta_0,\theta_1)$

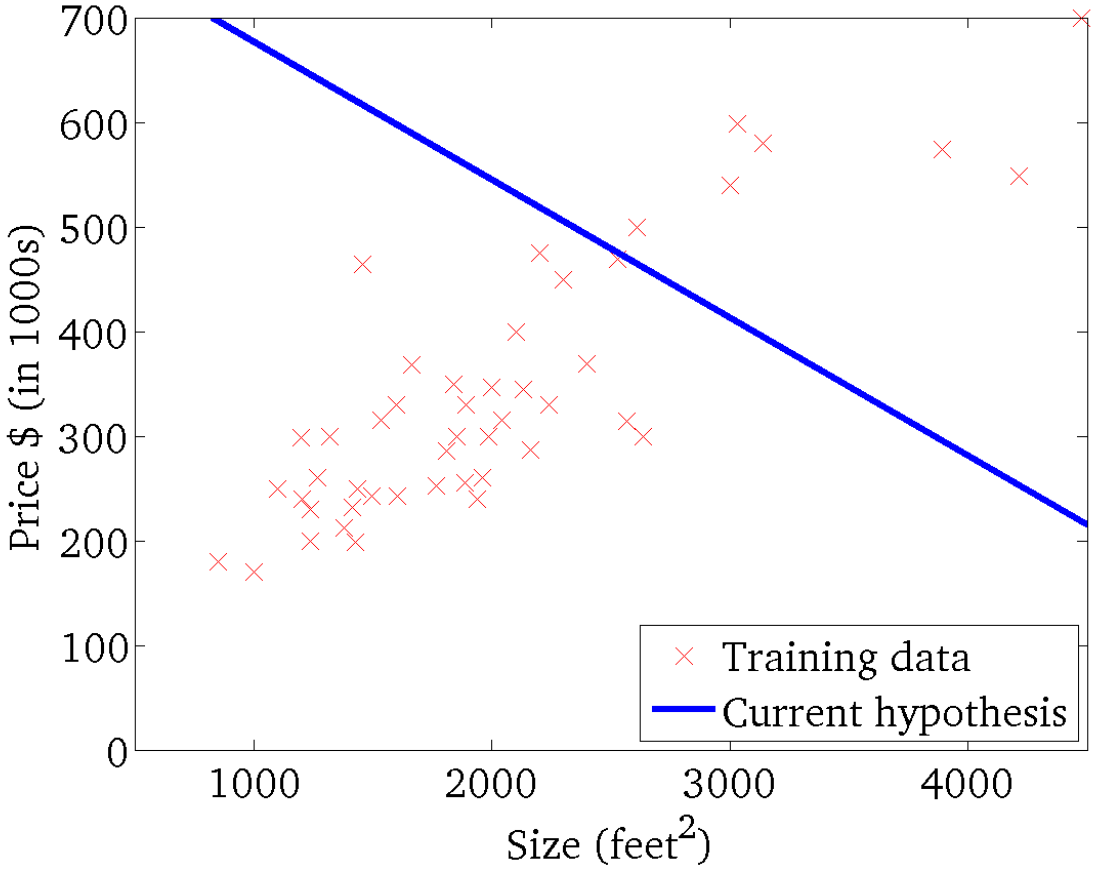




We can see that the height (y) indicates the value of the cost function, so find where y is at a minimum

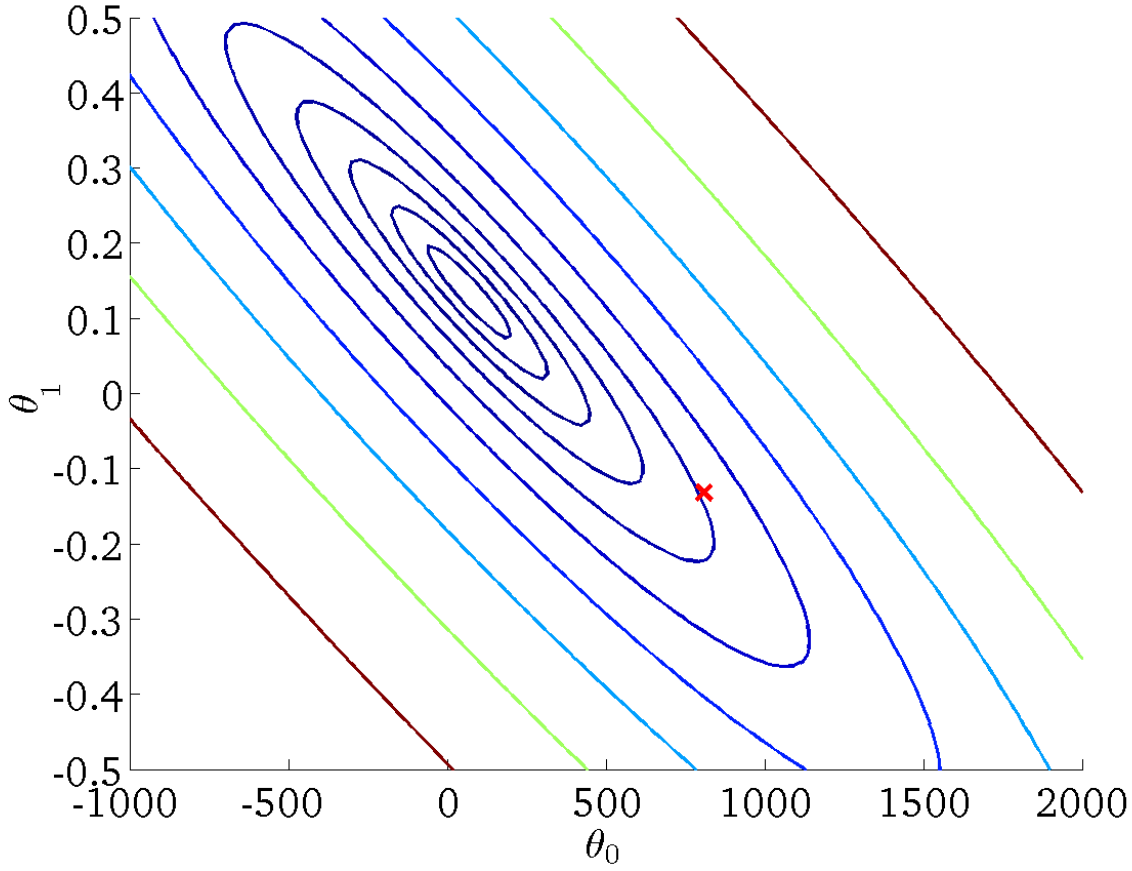
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



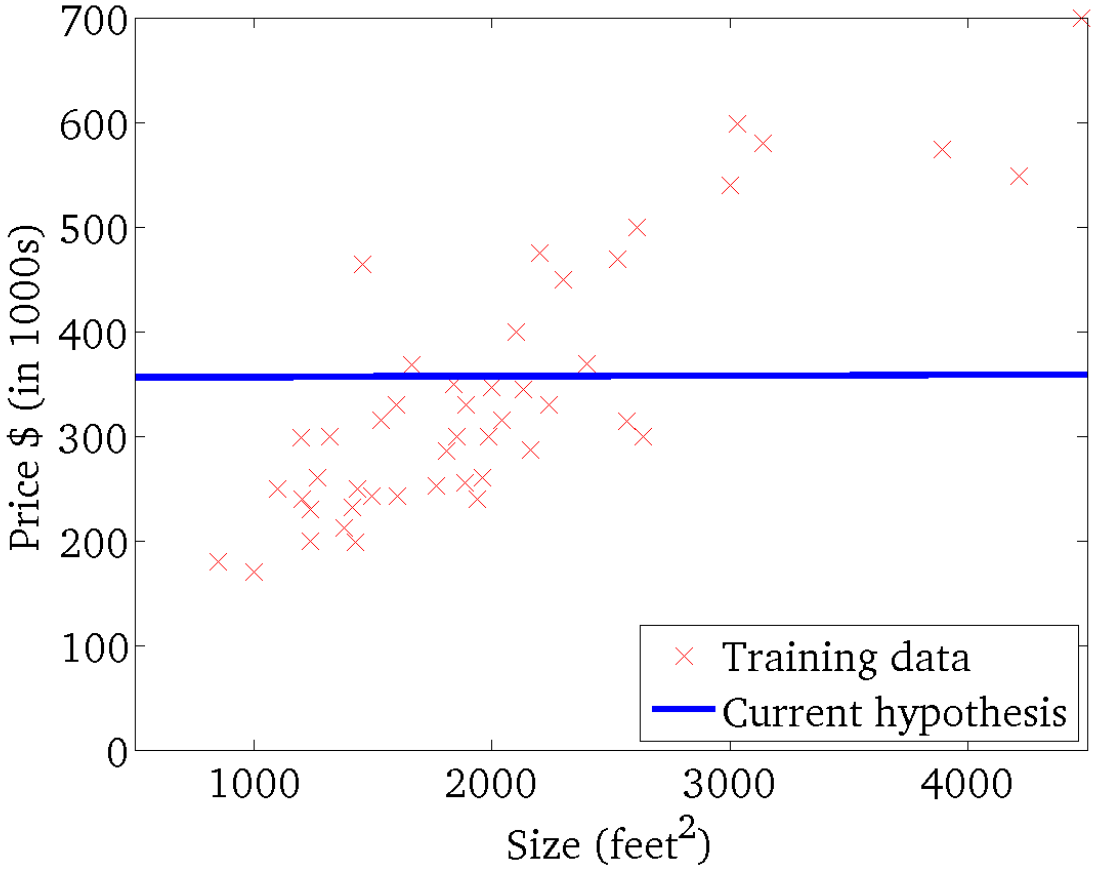
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



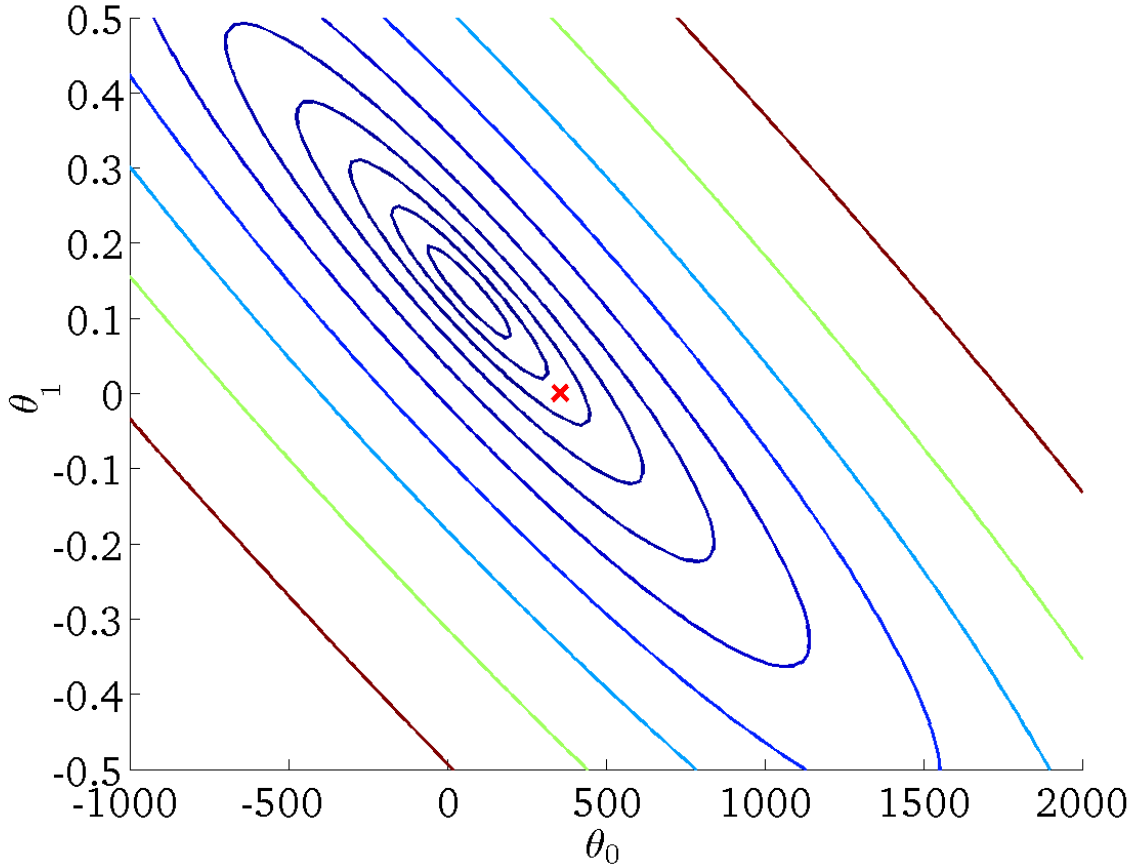
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



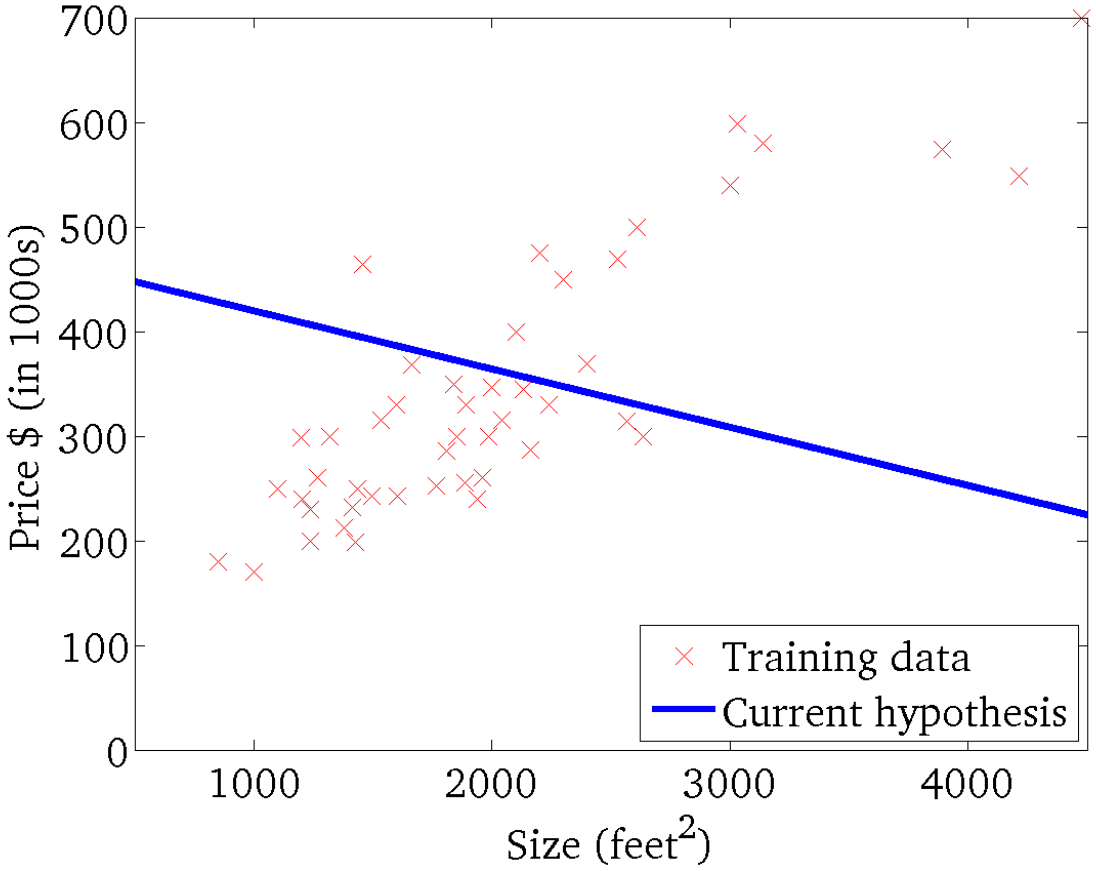
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



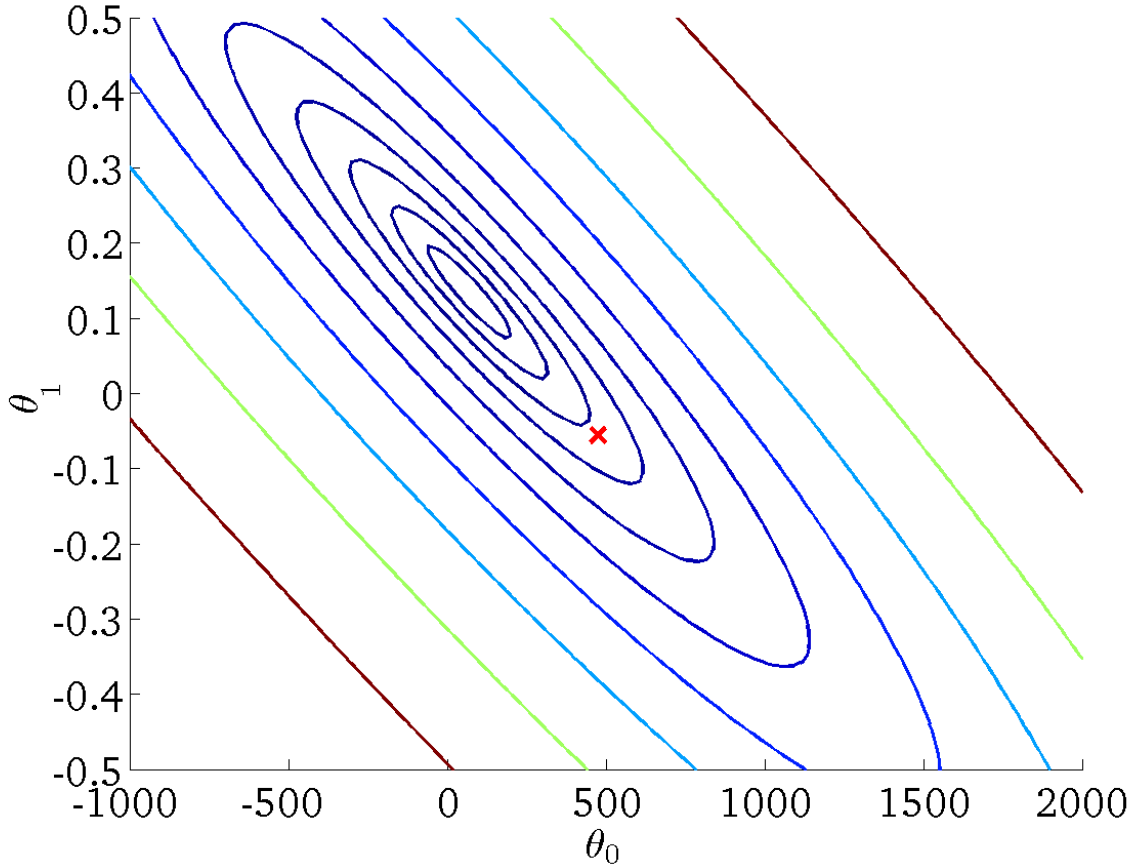
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



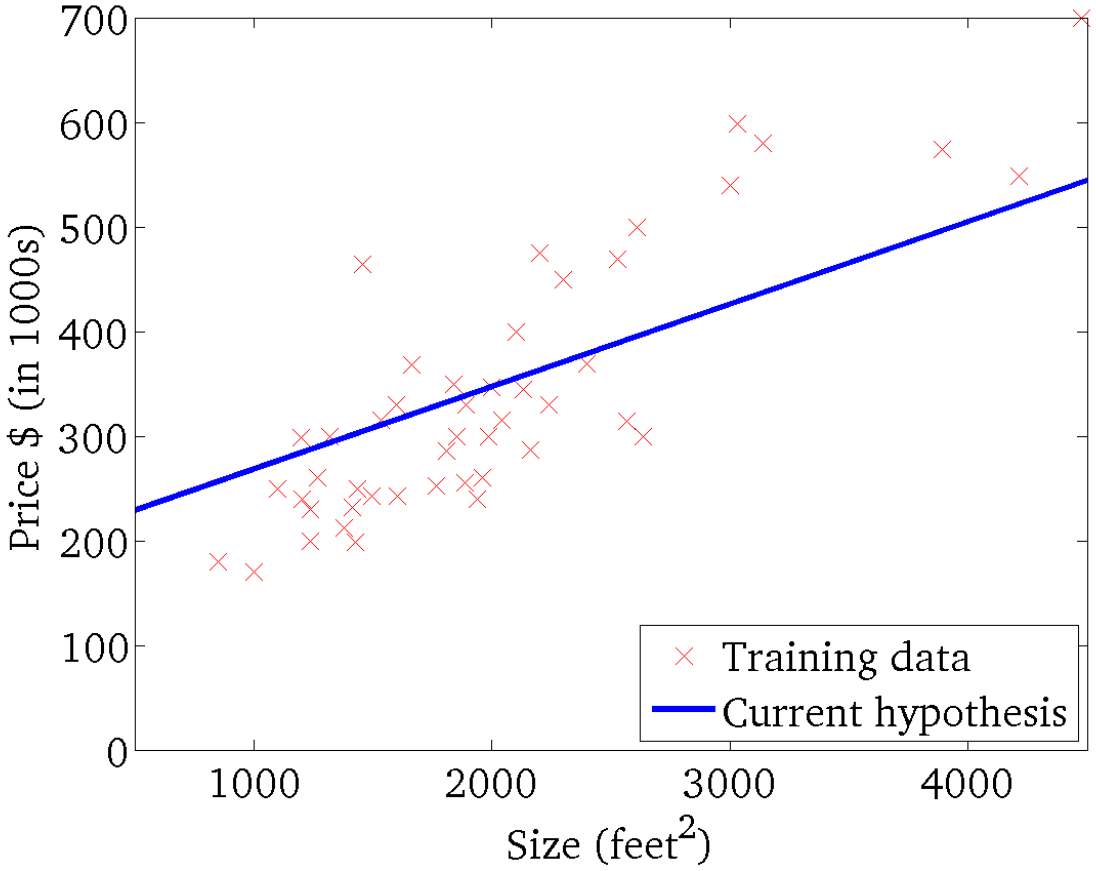
$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



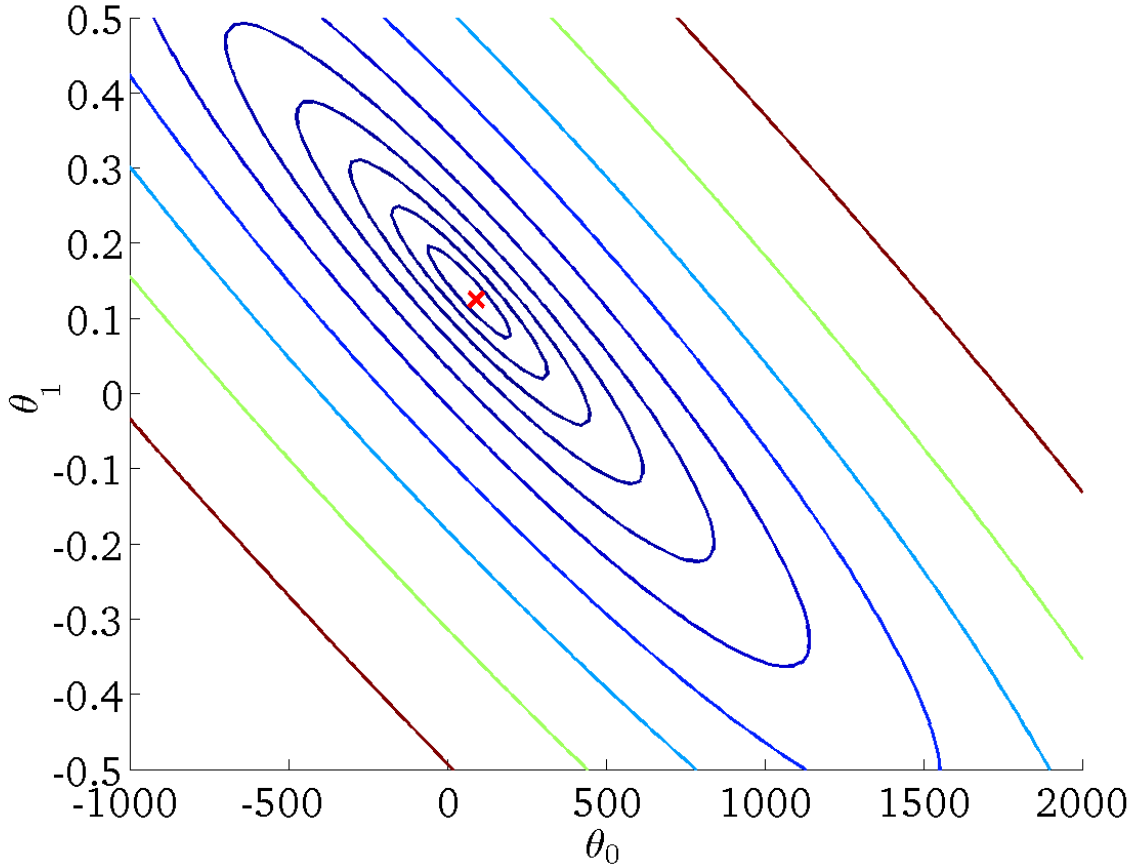
$h_\theta(x)$

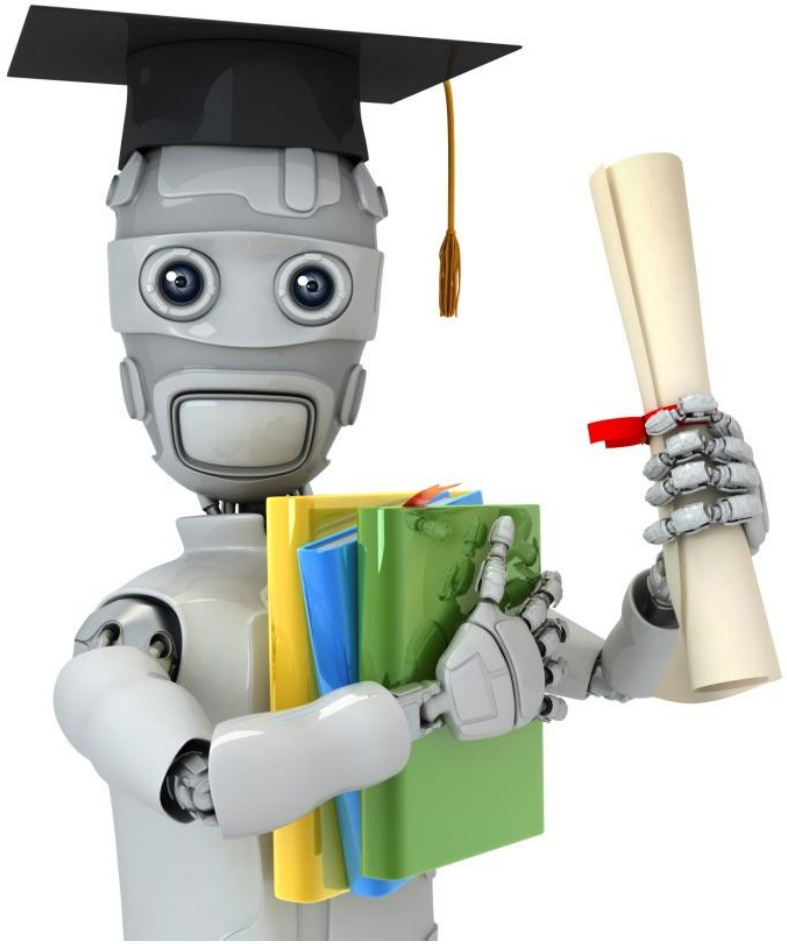
(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)





Machine Learning

Linear regression with one variable

Gradient descent

- Minimize cost function J
- Gradient descent
  - Used all over machine learning for minimization
- Start by looking at a general J() function
- Problem
  - We have $J(\theta_0, \theta_1)$
  - We want to get **min $J(\theta_0, \theta_1)$**

Have some function $J(\theta_0, \theta_1)$

Want $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Gradient descent applies to more general functions

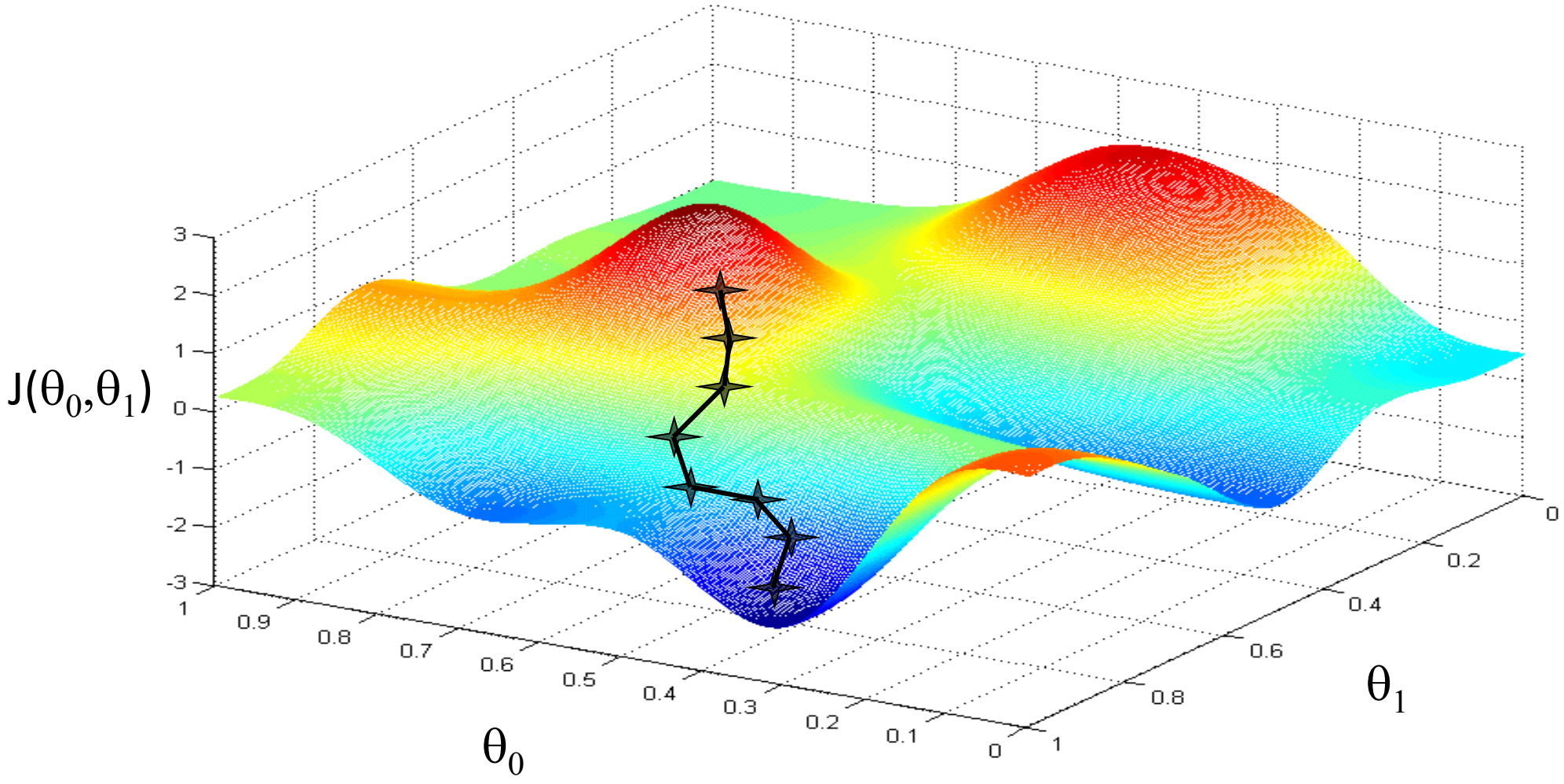
- ✓ $J(\theta_0, \theta_1, \theta_2 \ldots \theta_n)$
- ✓ $\min J(\theta_0, \theta_1, \theta_2 \ldots \theta_n)$

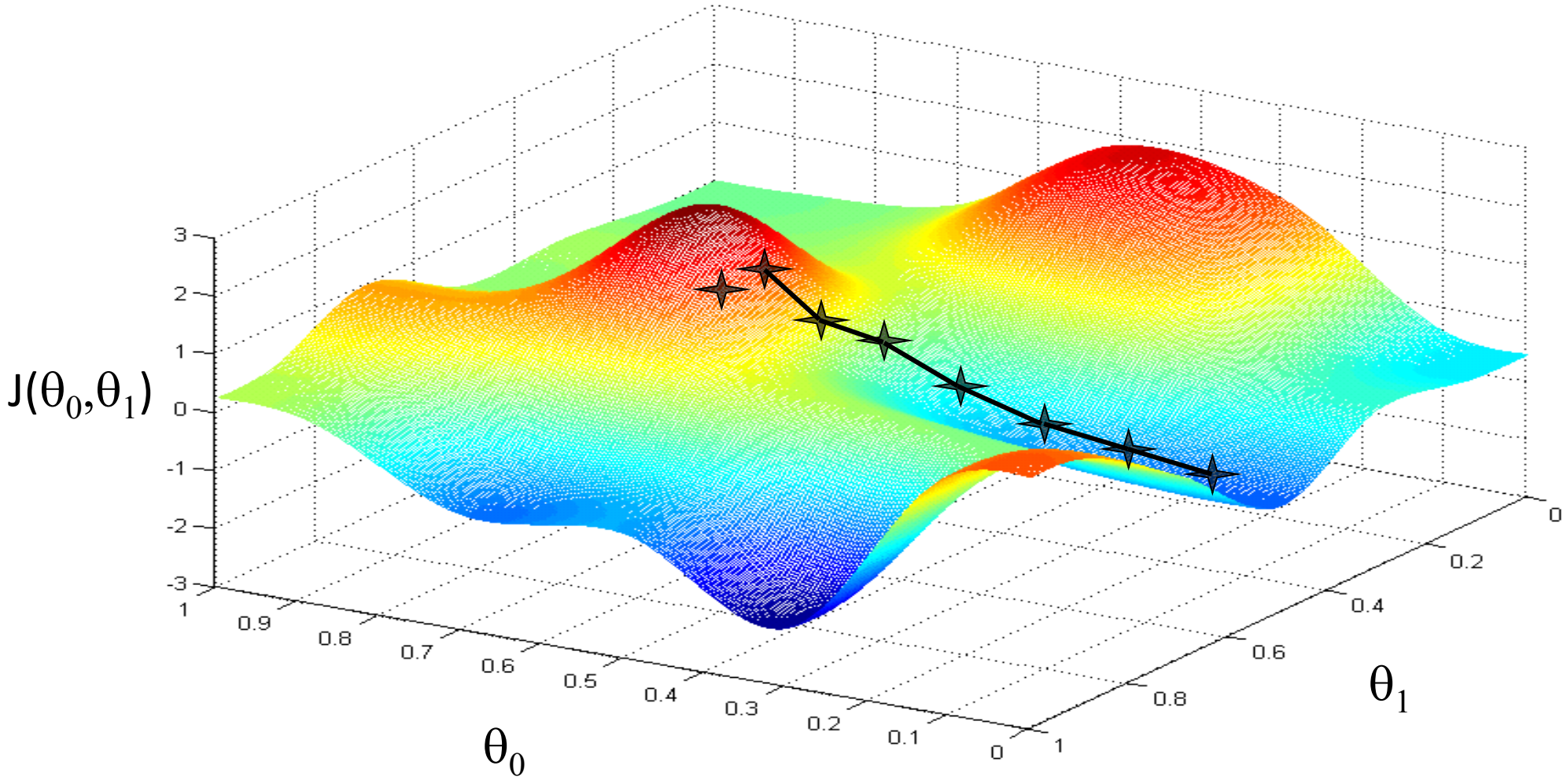
Outline:

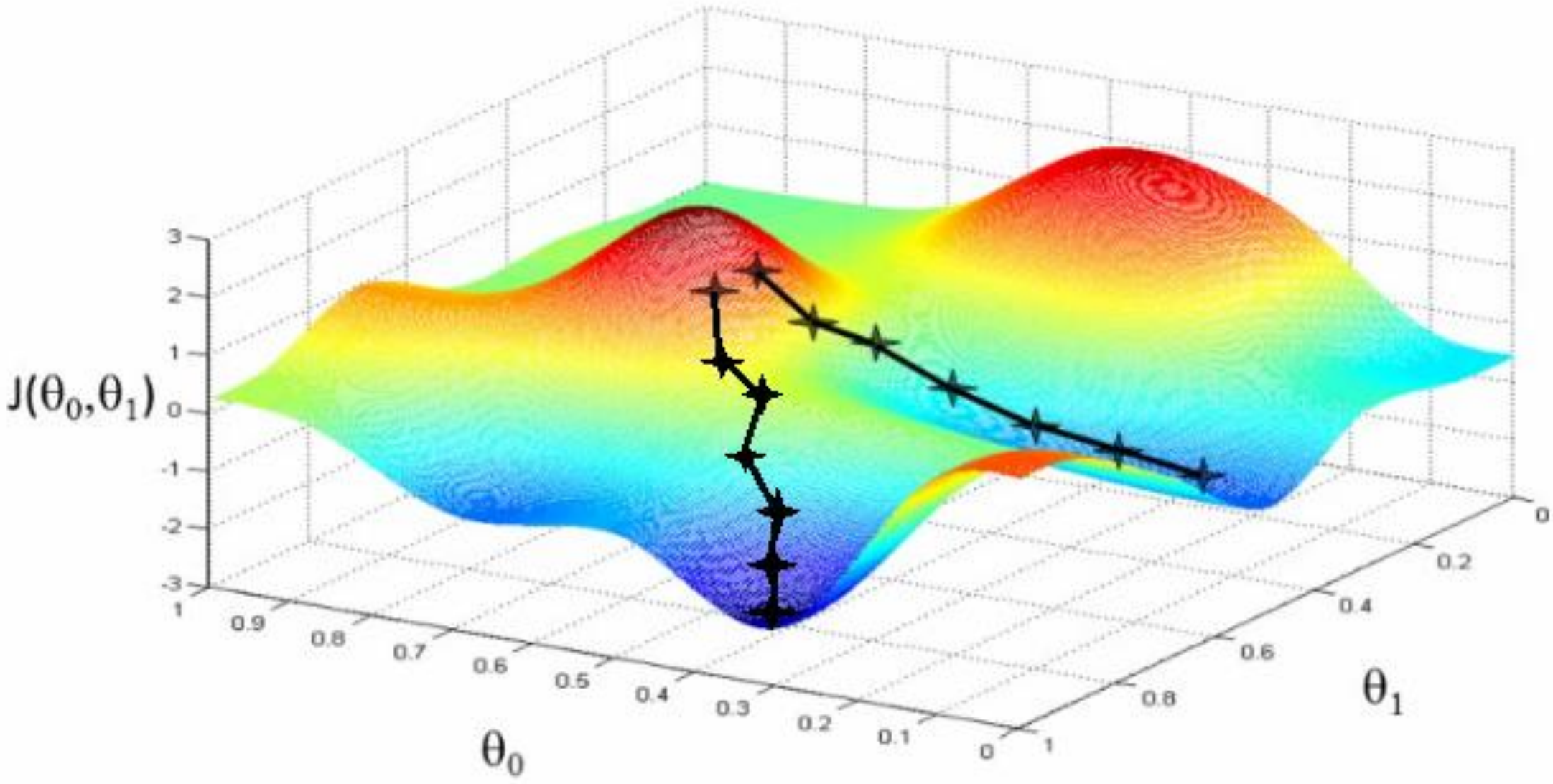
- Start with some θ_0, θ_1
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$

  until we hopefully end up at a minimum

- Start with initial guesses
  - Start at 0,0 (or any other value)
  - Keeping changing θ_0 and θ_1 a little bit to try and reduce $J(\theta_0,\theta_1)$
- Each time you change the parameters, you select the gradient which reduces $J(\theta_0,\theta_1)$ the most possible
- Repeat
- Do so until you converge to a local minimum
- Has an interesting property
  - Where you start can determine which minimum you end up
  - Here we can see one initialization point led to one local minimum
  - The other led to a different one


J(θ0,θ1)
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ0
0
0.2
0.4
0.6
0.8
1
θ1


$J(\theta_0,\theta_1)$
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ_0
0
0.2
0.4
0.6
0.8
1
θ_1


J(θ0,θ1)
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ0
0
0.2
0.4
0.6
0.8
1
θ1

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \qquad (\text{for } j = 0 \text{ and } j = 1)$$

}

Correct: Simultaneous update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\theta_1 := \text{temp1}$$

Incorrect:

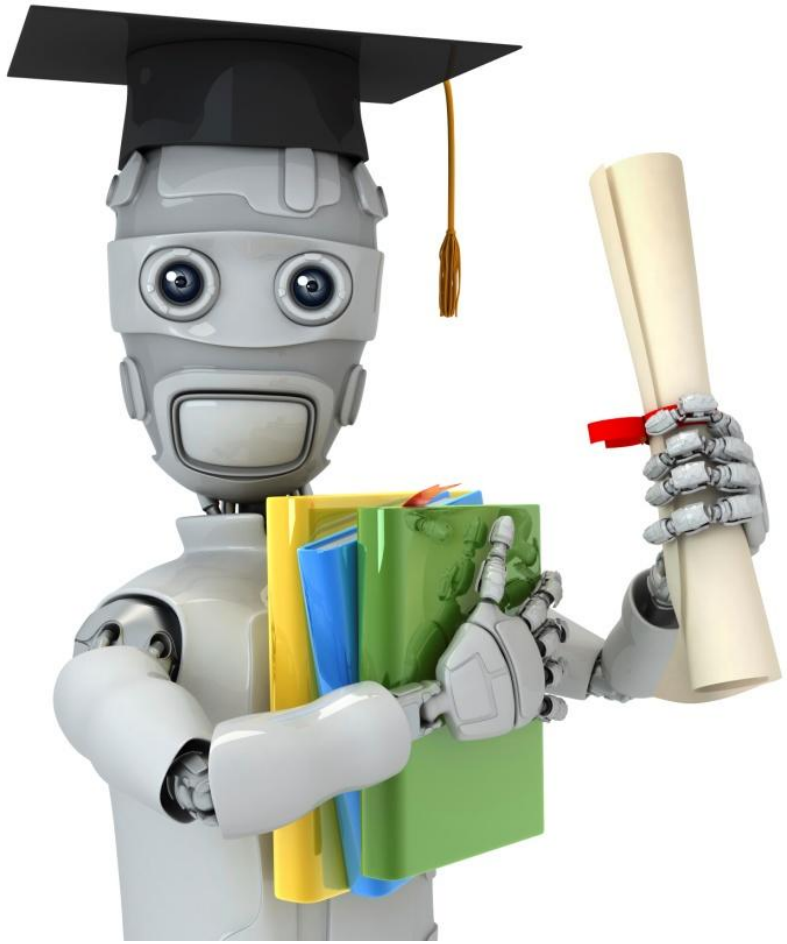
$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_1 := \text{temp1}$$

Notation:=

- Denotes assignment
- NB a = b is a *truth assertion*

α (alpha)

- Is a number called the learning rate
- Controls how big a step you take
  - If α is big have an aggressive gradient descent
  - If α is small take tiny steps



Machine Learning

Linear regression with one variable

Gradient descent intuition

Gradient descent algorithm

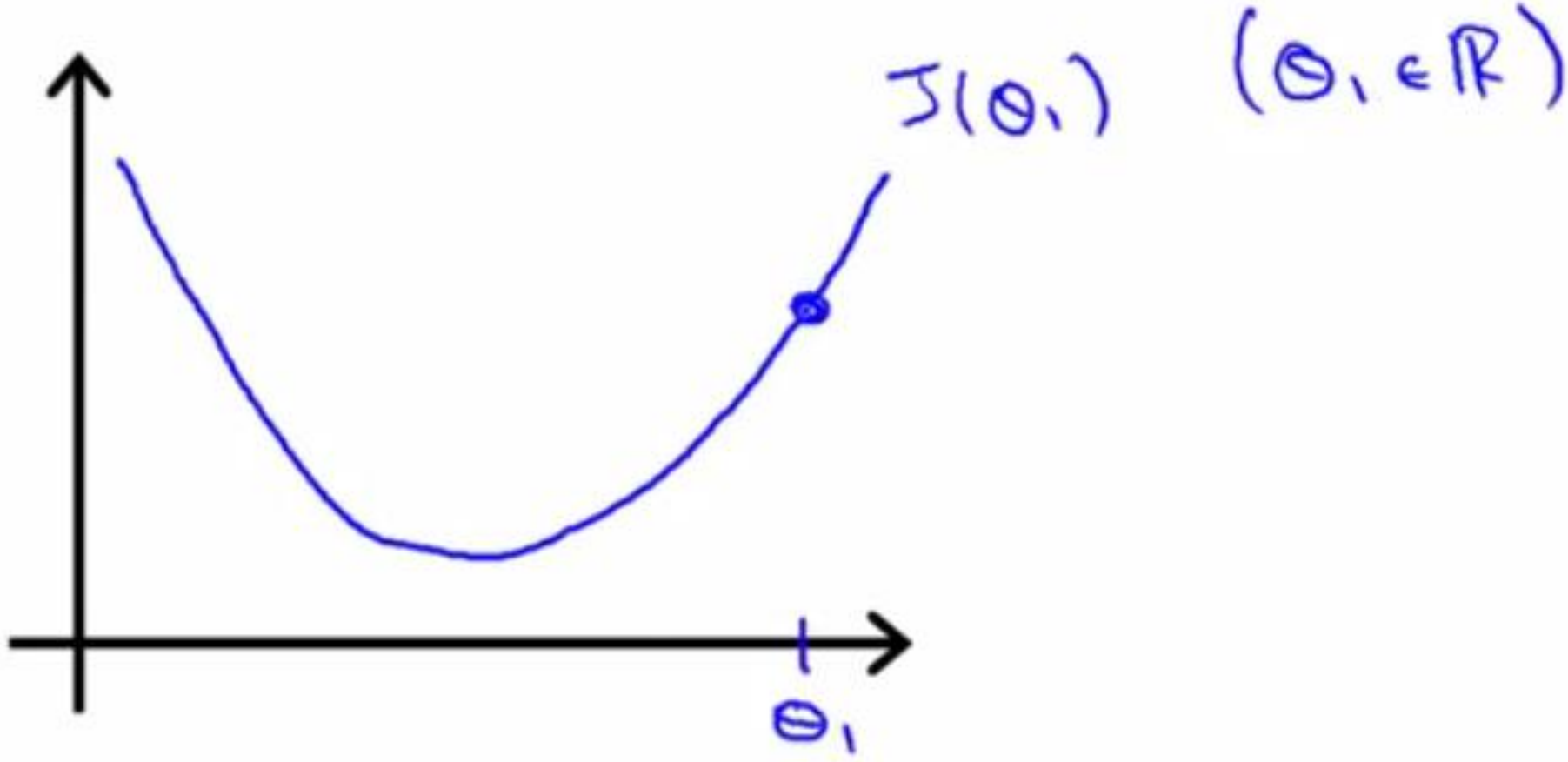
$$\text{repeat until convergence } \{$$
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \qquad \begin{matrix}(\text{simultaneously update} \\ j = 0 \text{ and } j = 1)\end{matrix}$$
$$\}$$

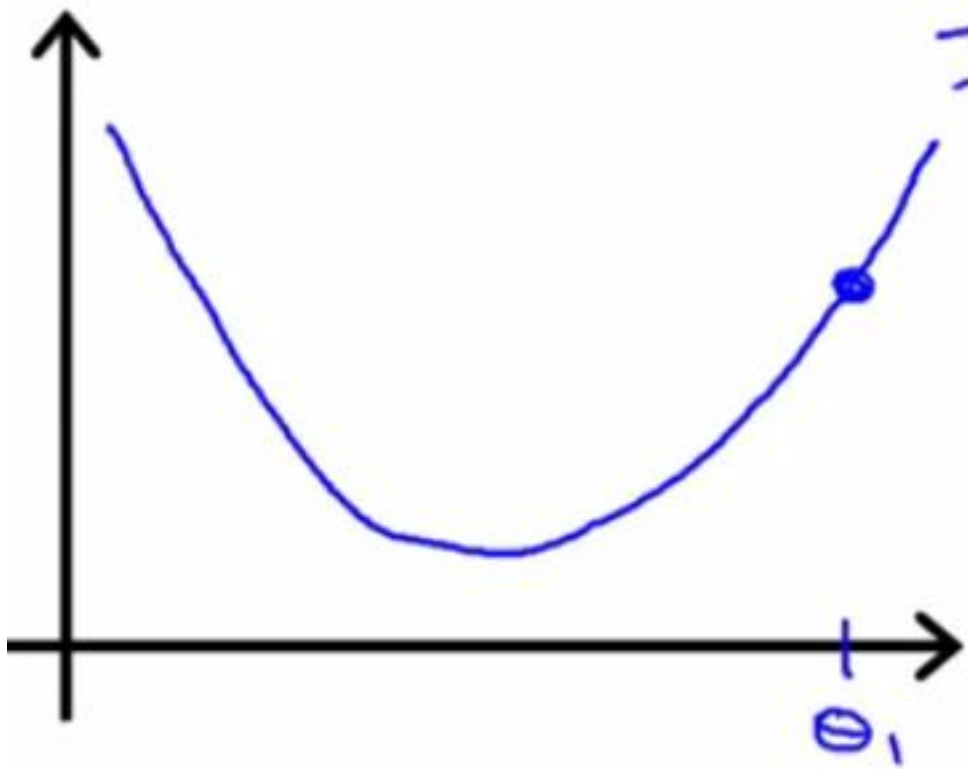
To understand gradient descent, we'll return to a simpler function where we minimize one parameter to help explain the algorithm in more detail.

- min θ_1 $J(\theta_1)$ where θ_1 is a real number

Notation nuances

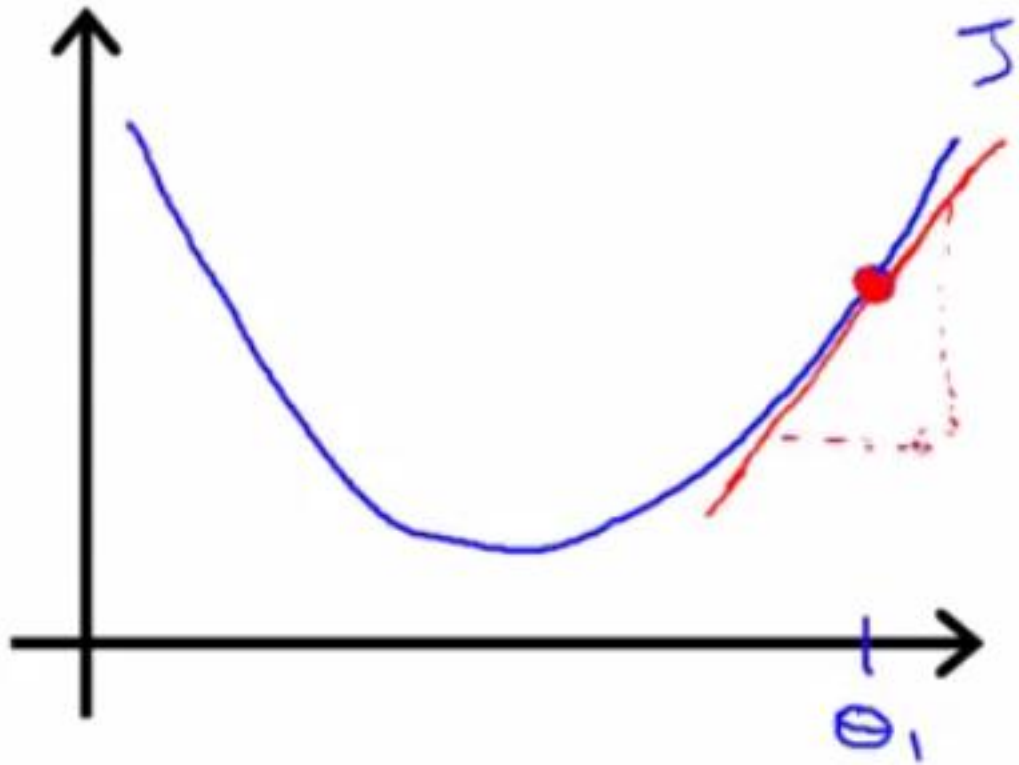
- Partial derivative vs. derivative
  - Use partial derivative when we have multiple variables but only derive with respect to one
  - Use derivative when we are deriving with respect to all the variables


$J(\theta_1)$
$(\theta_1 \in \mathbb{R})$
θ_1



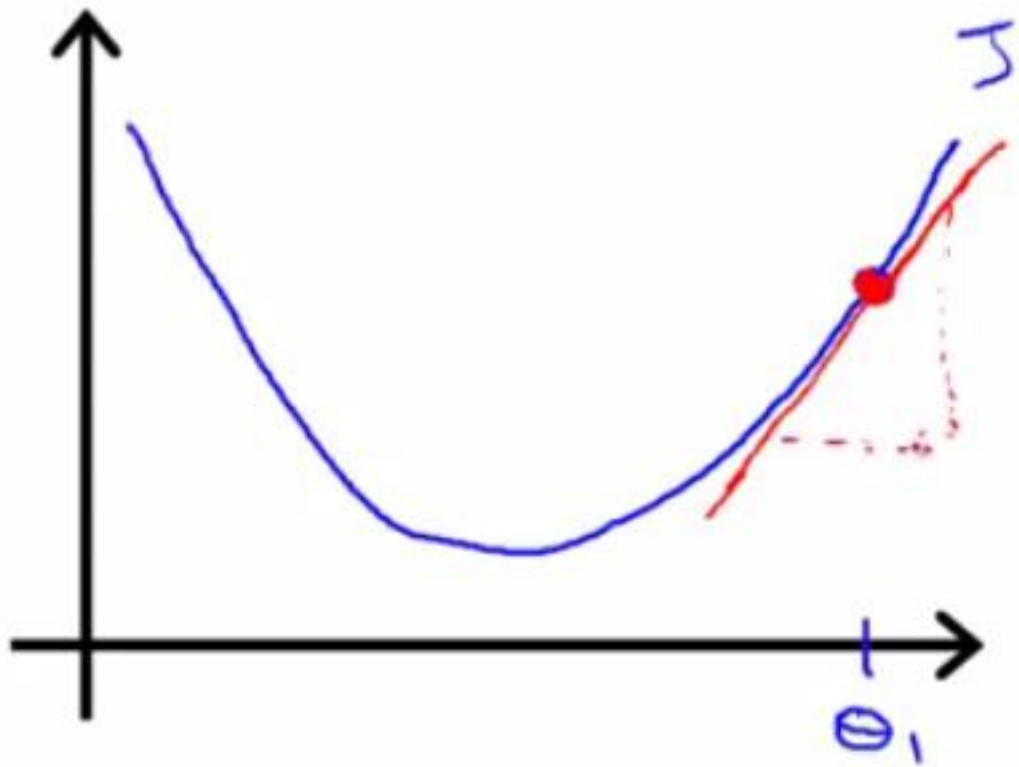
$J(\theta_1)$ $(\theta_1 \in \mathbb{R})$

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$



$J(\theta_1)$ $(\theta_1 \in \mathbb{R})$

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

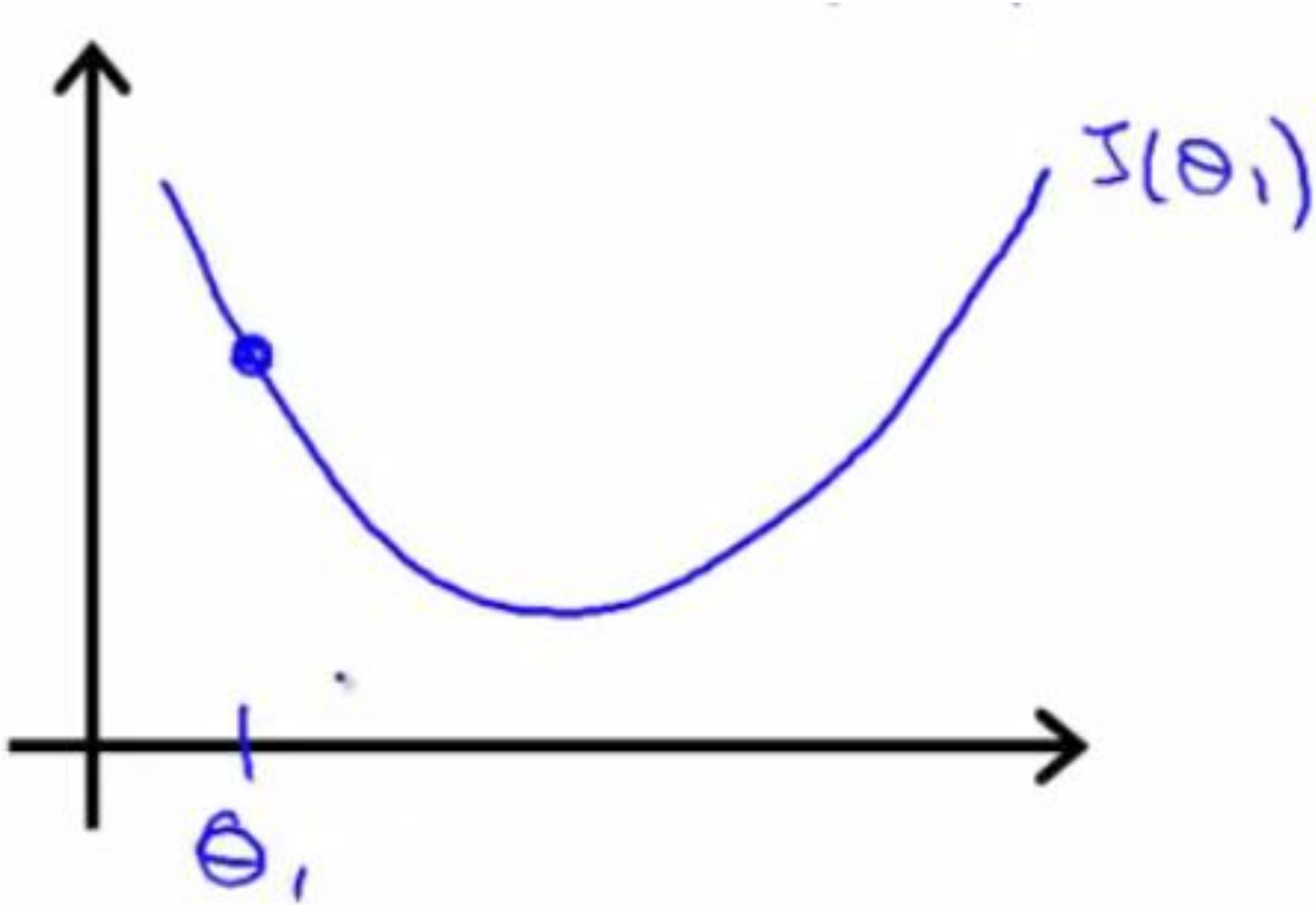


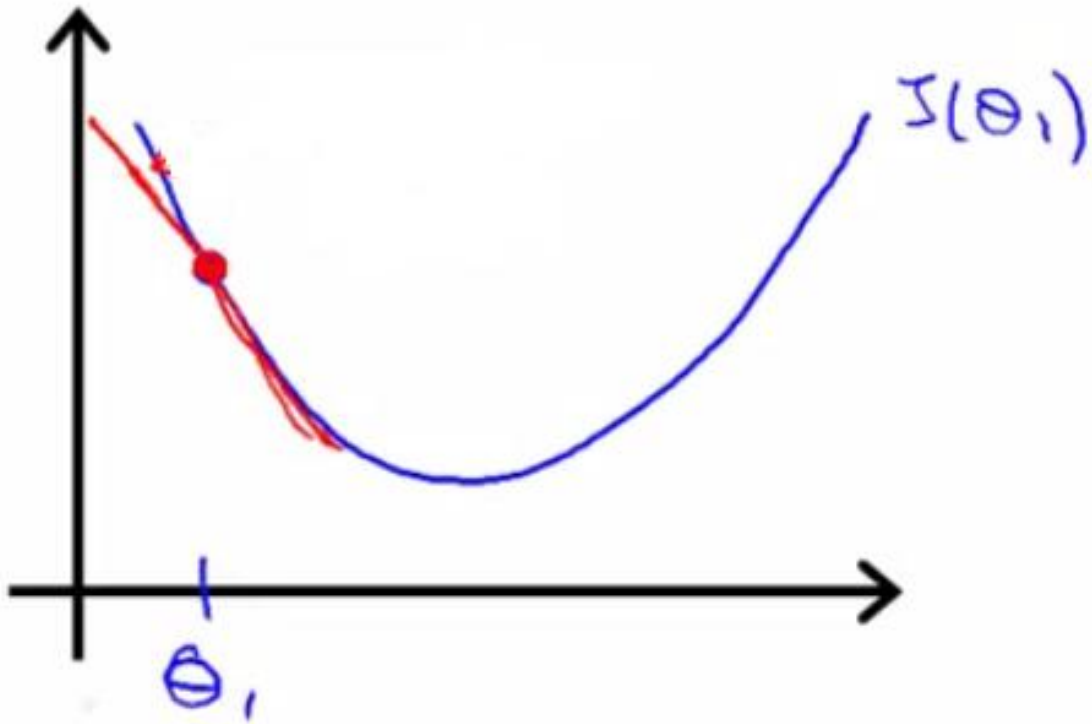
$J(\theta_1)$ $(\theta_1 \in \mathbb{R})$

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

$$\geq 0$$

$\theta_1 := \theta_1 - \alpha \cdot$ (positive number)


J(θ1)
θ1



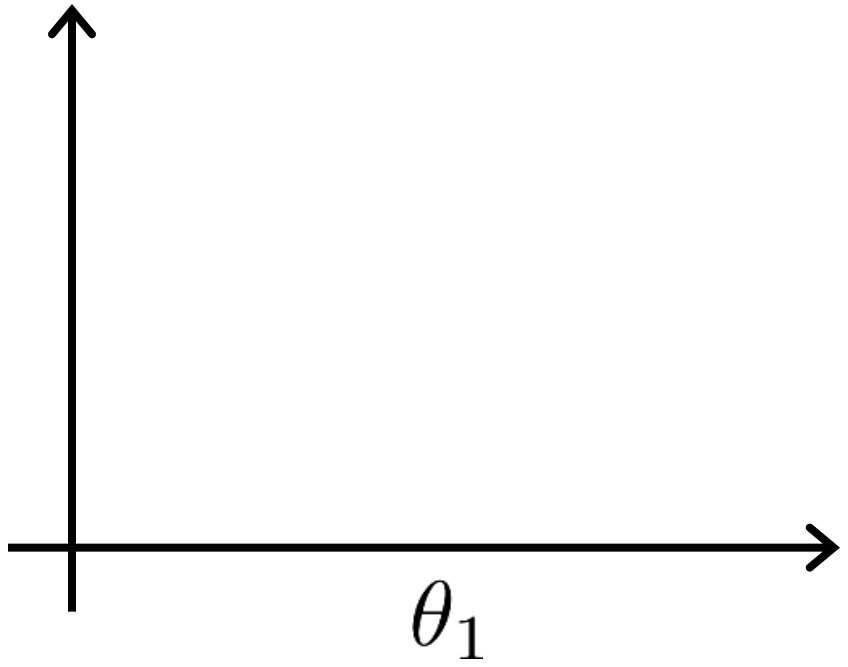
$$\frac{d}{d\theta_1} J(\theta_1)$$

$$\leq 0$$

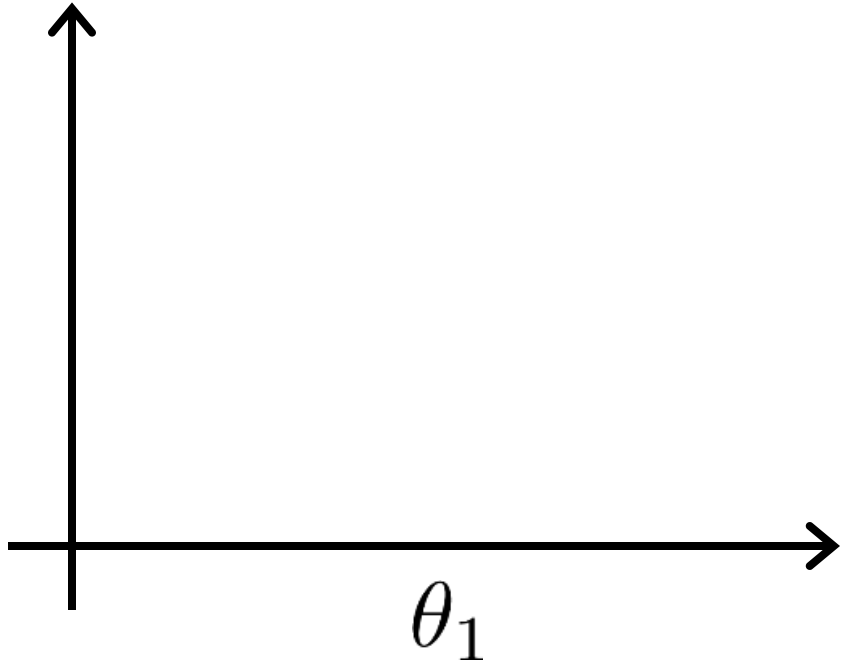
$$\theta_1 := \theta_1 - \alpha \text{ (negative number)}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

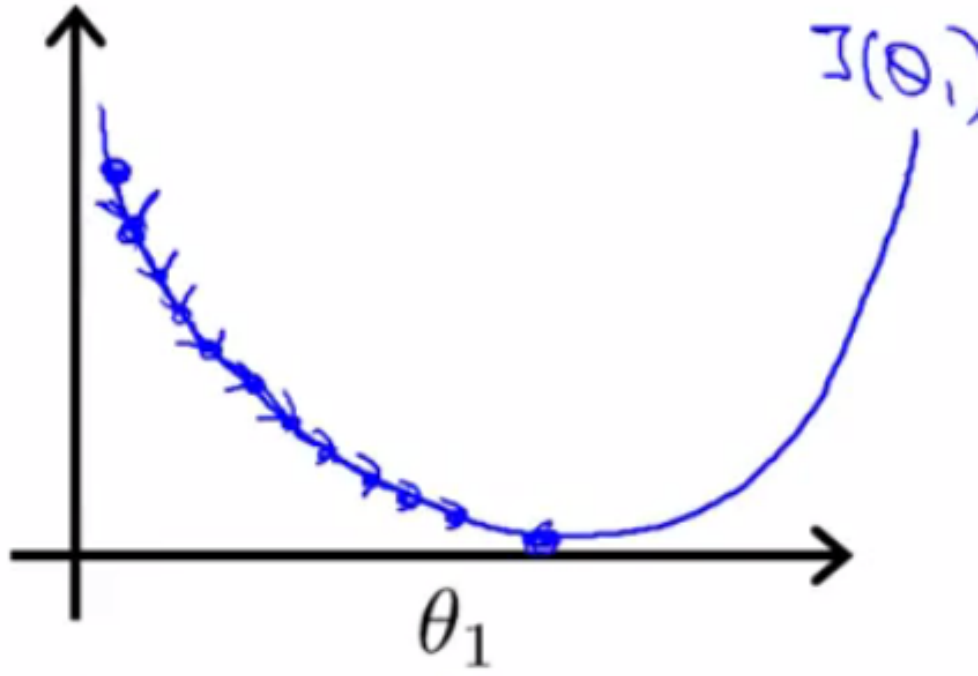


If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

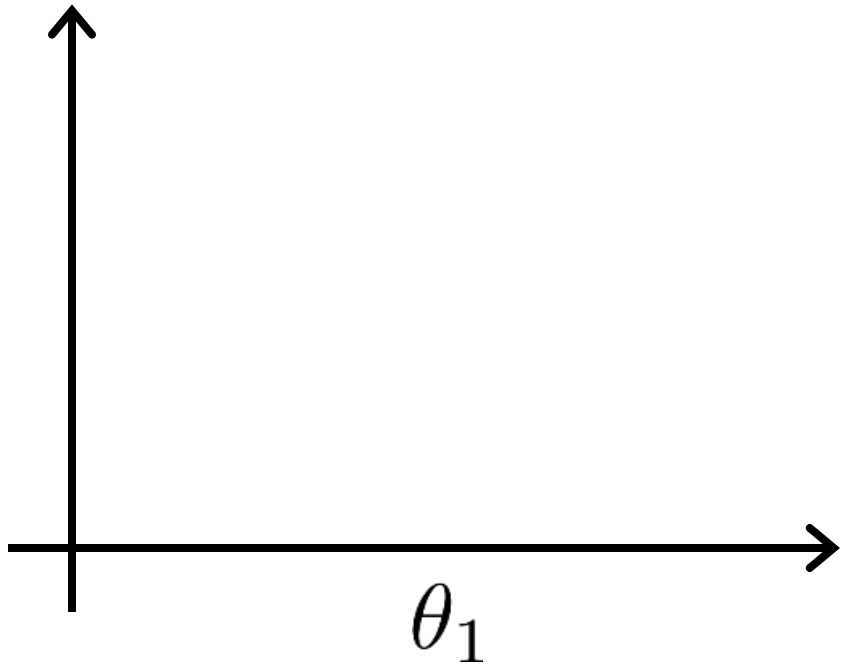


$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.



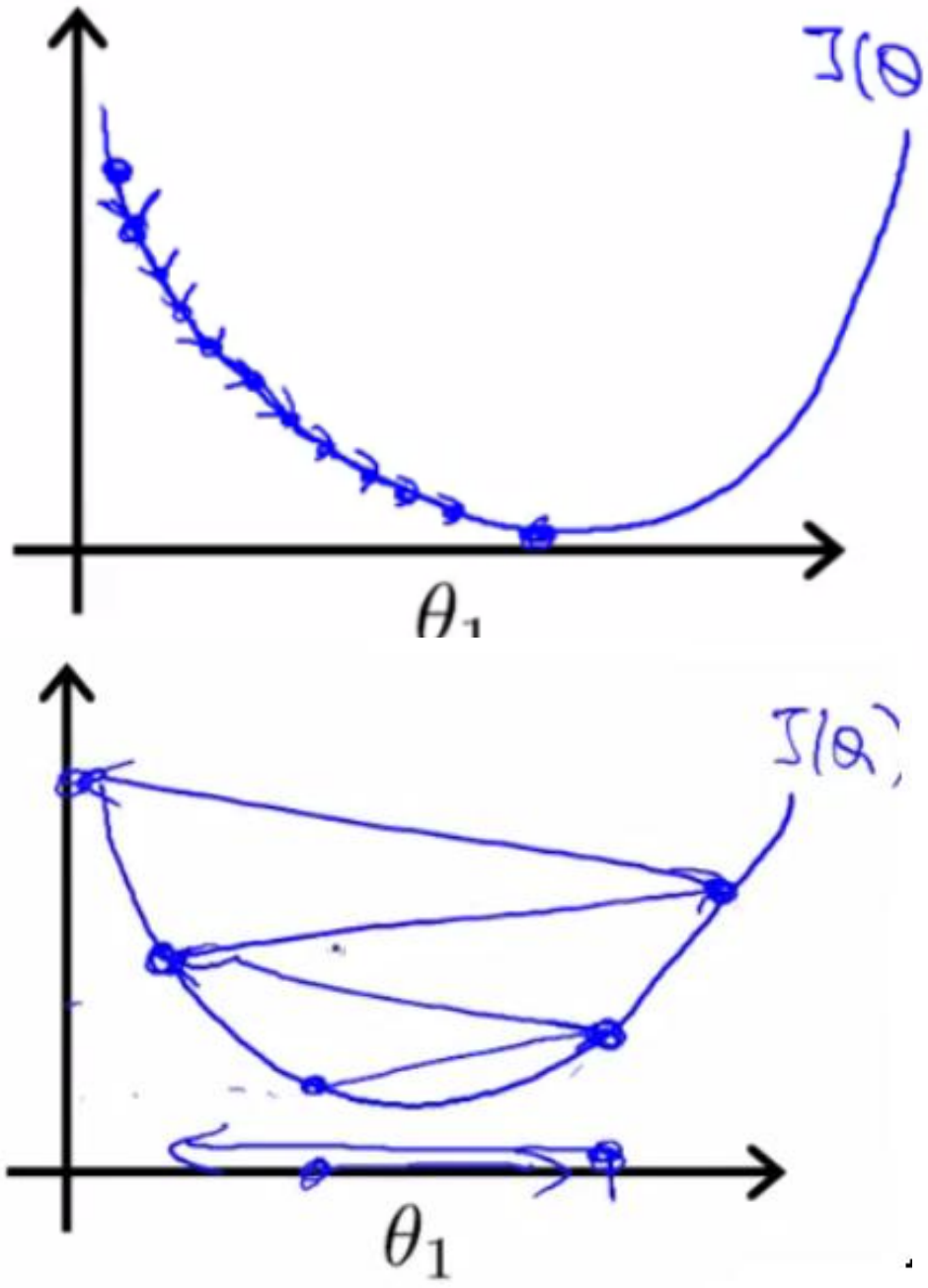
If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

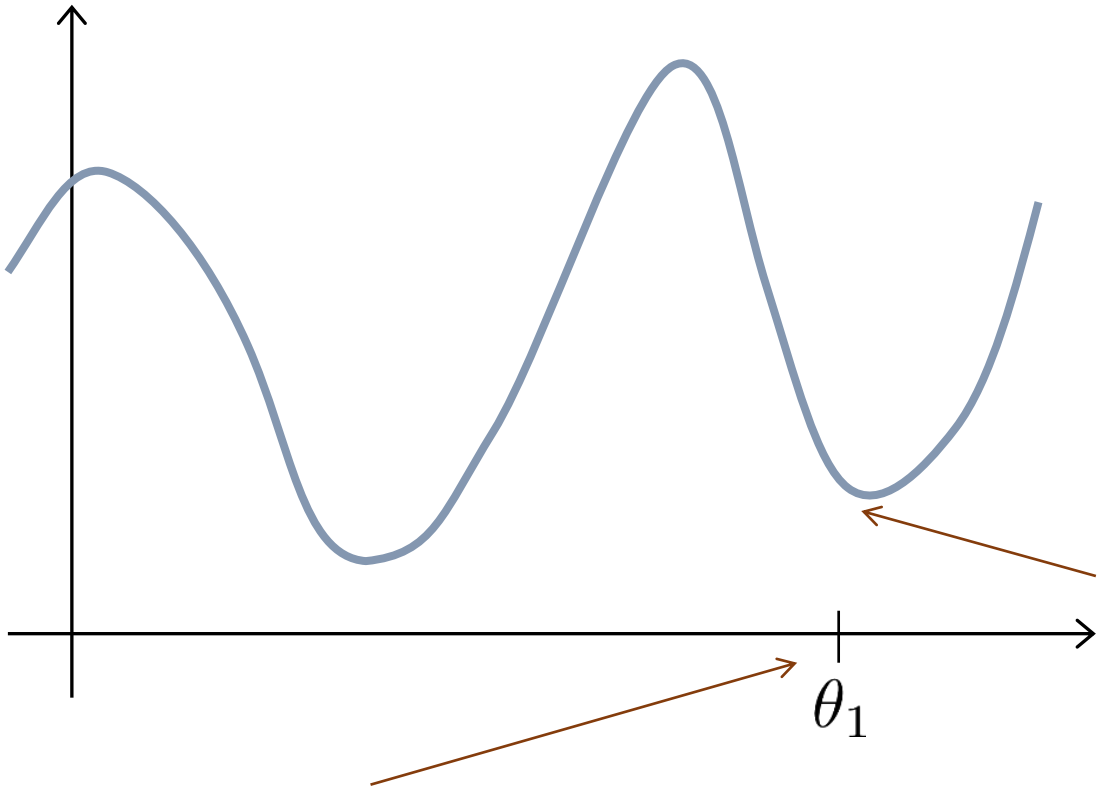


$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.





θ_1 at local optima

Current value of θ_1

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

$\theta_1 := \theta_1 - \alpha \cdot 0$

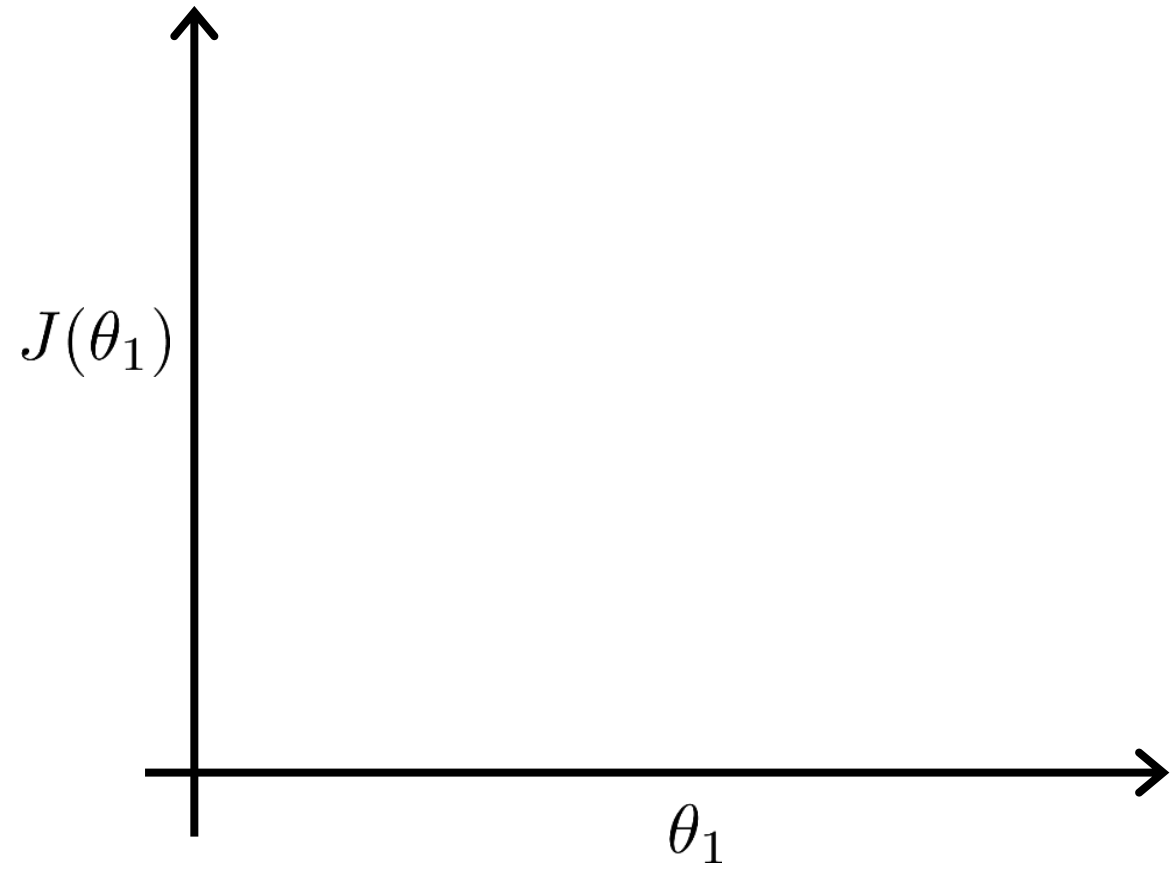
$\theta_1 := \theta_1$

- Alpha term (α)
  - What happens if alpha is too small or too large
  - Too small
    - Take baby steps
    - Takes too long
  - Too large
    - Can overshoot the minimum and fail to converge
- When you get to a local minimum
  - Gradient of tangent/derivative is 0
  - So derivative term = 0
  - alpha * 0 = 0
  - So $\theta_1 = \theta_1 - 0$
  - So θ_1 remains the same

Gradient descent can converge to a local minimum, even with the learning rate α fixed.

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

As we approach a local minimum, gradient descent will automatically take smaller steps. So, no need to decrease α over time.



As you approach the global minimum the derivative term gets smaller, so your update gets smaller, even with alpha is fixed

- Means as the algorithm runs you take smaller steps as you approach the minimum
- So no need to change alpha over time

Now derive the formula for the weight updates for gradient descent algorithm